EE529: Vision Based Control

Term Project

Title:

Implementation and Analysis of Adaptive Switch Image-Based Visual Servoing: Reproducing the Method of Ghasemi et al.

Name: Gizem Doğa Filiz

ID: 29450

Submission Date: 12/01/2025

Faculty of Engineering and Natural Sciences

Program: Mechatronics Engineering

Sabancı Üniversitesi

Abstract

This project presents the MATLAB-based implementation and analysis of the adaptive switch Image-Based Visual Servoing (IBVS) controller proposed by Ghasemi et al. (2020). The original paper's methodology is reproduced, including the staged control approach that decouples rotational and translational movements to enhance system response. Stability analysis based on the Lyapunov method is provided to verify the controller's robustness. A key focus is on overcoming inherent drawbacks of conventional IBVS, such as its inability to perform a 180° camera rotation around its center. While the implementation focuses on kinematic control rather than torque-based dynamics, key principles and theoretical guarantees are maintained. The results demonstrate the controller's effectiveness in feature tracking and convergence behavior within a simulated environment.

1 Introduction

Visual Servoing is a pivotal technique in robotic control, leveraging visual information to guide and manipulate the movement of robotic systems. Among its various implementations, Image-Based Visual Servoing (IBVS) stands out due to its direct use of image features for control, offering robustness in dynamic and uncertain environments. The seminal paper titled **"Adaptive Switch Image-based Visual Servoing for Industrial Robots"** by Ahmad Ghasemi, Pengcheng Li, and Wen-Fang Xie, published in the *International Journal of Control, Automation and Systems* (Vol. 18, No. 5, 2020), presents a significant advancement in the field of IBVS for industrial robotics.

1.1 Overview of the Paper

Ghasemi et al. introduce an **Adaptive Switch IBVS controller** tailored for industrial robots operating in monocular eye-in-hand configurations. The core innovation lies in decoupling the rotational and translational movements of the camera, thereby decomposing the IBVS control process into three distinct stages, each governed by different control gains. This decomposition not only accelerates the system's response but also enhances the tracking performance. Furthermore, the proposed adaptive law addresses uncertainties inherent in monocular camera parameters, a critical factor often overlooked in traditional IBVS implementations. The stability of the controller is rigorously validated using the Lyapunov method, ensuring reliability in industrial applications. Experimental validations conducted on a 6-degree-of-freedom (DOF) Denso robot demonstrate the controller's superior performance compared to conventional IBVS methods, particularly in terms of response time and tracking accuracy. Additionally, the controller effectively mitigates some of IBVS's intrinsic limitations, such as the inability to execute a 180° rotation around the camera center.

1.2 Aims and Problem Statement

The primary aim of Ghasemi et al.'s research is to develop a robust and efficient IBVS controller suitable for industrial robotic applications, where rapid and precise movements are paramount. Traditional IBVS systems, while effective in feature tracking, often suffer from prolonged convergence times and susceptibility to image singularities, especially when initial feature configurations are distant from desired states. These limitations

render them less ideal for high-speed industrial tasks that demand quick responsiveness and reliability.

To address these challenges, the paper proposes an **adaptive switch control scheme** that:

1. **Decouples Camera Movements**: By separating rotational and translational commands, the system can apply specialized control gains tailored to each movement type, thereby optimizing performance.

2. **Enhances System Response**: The introduction of multiple control stages with varying gains significantly reduces the system's response time, making it more suitable for real-time industrial applications.

3. **Handles Camera Uncertainties**: The adaptive law incorporated into the controller compensates for uncertainties in camera parameters, such as focal length and principal point coordinates, which are critical for accurate feature tracking.

4. **Ensures Stability**: The use of the Lyapunov method provides a theoretical foundation for the controller's stability across all operational stages, a feature not extensively covered in previous switch control schemes.

1.3 Differences Between Implementation and Original Paper

While the foundational principles of IBVS are retained, my implementation diverges from Ghasemi et al.'s original methodology in several key aspects, primarily due to practical considerations and available resources:

1. **Simulated Environment vs. Real-World Experimentation**:

   - **Original Paper**: Conducted experiments using a real-world setup with a 6-DOF Denso robot and a physical monocular camera, allowing for practical validation of the controller's performance in actual industrial scenarios.
   - **My Implementation**: Utilizes a simulated environment within MATLAB, employing a simulated camera model. This approach facilitates easier manipulation and observation of variables but may not capture all real-world complexities.

2. **Robot Platform**:

   - **Original Paper**: Employed a 6-DOF Denso robot, a common choice in industrial settings due to its precision and reliability.
   - **My Implementation**: Utilizes a 6-DOF Universal Robots UR5 model, which, while similar in degrees of freedom, may have different dynamic properties and kinematic configurations.

3. **Dynamic Considerations and Torque Control**:

   - **Original Paper**: Incorporated the dynamics of the robot, including torque control, to achieve more refined and stable movements. This consideration allows for better handling of forces and torques, leading to enhanced control performance.

- **My Implementation**: Omitted the dynamic aspects and torque control due to the unavailability of exact dynamic models for the Universal UR5 robot. Consequently, the implementation focuses on kinematic control, which simplifies the control strategy but may sacrifice some performance nuances achievable through dynamic modeling.

4. **Adaptive Law and Regression Matrix Calculation**:
   - **Original Paper**: Implemented an adaptive law that accounts for uncertainties in camera parameters by calculating a regression matrix and updating estimates of camera parameters over time.
   - **My Implementation**: Skipped the regression matrix calculation and the adaptive updating of uncertain camera parameters ($\hat{Q}(t)$ and $\hat{Q}_i(t)$). This simplification arises from the lack of precise dynamic models and the focus on a simulated setup, thereby reducing computational complexity at the expense of adaptability to parameter uncertainties.
5. **Control Signal Generation**:
   - **Original Paper**: Generated control signals based on torque commands, directly influencing the robot's joint torques for movement.
   - **My Implementation**: Generates velocity commands at the kinematic level, adjusting joint velocities to achieve the desired end-effector movements. This approach aligns with the simulated environment's constraints but deviates from the torque-based control in the original study.

Despite these differences, the essence of adaptive switch IBVS is preserved, aiming to enhance response speed and tracking accuracy while mitigating some inherent IBVS drawbacks.

2 Methodology

This section elucidates the methodological framework employed in the original paper by Ghasemi et al. (2020) and delineates the adapted approach utilized in my MATLAB-based implementation. The discussion encompasses the fundamental equations governing IBVS, the design of the adaptive switch controller, and the mathematical models underpinning the control strategy.

2.1 Image-Based Visual Servoing (IBVS) Fundamentals

IBVS leverages image features extracted from a camera to guide the robot's end-effector towards a desired position. The process involves comparing current image features with desired features and generating control signals to minimize the error between them.

2.1.1 Image Feature Representation

Let $s_i(t) = \begin{bmatrix} x_i(t) \\ y_i(t) \end{bmatrix}$ denote the image coordinates of the i-th feature at time t, and $s_{d_i} = \begin{bmatrix} x_{d_i} \\ y_{d_i} \end{bmatrix}$ represent the desired image coordinates for the same feature. For n features, these

are aggregated as:

$$s(t) = \begin{bmatrix} s_1(t) \\ \vdots \\ s_n(t) \end{bmatrix}, \quad s_d = \begin{bmatrix} s_{d1} \\ \vdots \\ s_{dn} \end{bmatrix} \tag{1}$$

2.1.2 Image Feature Velocity and Camera Velocity Relationship

The velocity of the image features, $\dot{s}(t)$, is related to the camera velocity, $V_c(t)$, through the image Jacobian matrix $J_{img}(t)$:

$$\dot{s}(t) = J_{img}(t) V_c(t) \tag{2}$$

where $V_c(t)$ is a 6×1 screw velocity vector comprising translational $V_{ct}(t)$ and rotational $V_{cr}(t)$ components:

$$V_c(t) = \begin{bmatrix} V_{ct}(t) \\ V_{cr}(t) \end{bmatrix} \in \mathbb{R}^{6\times 1} \tag{3}$$

2.1.3 Image Jacobian Matrix

The image Jacobian $J_{img}(t)$ encapsulates the relationship between camera motion and image feature movement. For a monocular eye-in-hand configuration, assuming all features have the same depth $Z(t)$, the Jacobian for each feature i is given by:

$$J_{img}(s_i(t)) = \begin{bmatrix} \frac{f}{Z(t)} & 0 & -\frac{x_i(t)}{Z(t)} & -\frac{x_i(t)y_i(t)}{f} & \frac{f^2+x_i(t)^2}{f} & -y_i(t) \\ 0 & \frac{f}{Z(t)} & -\frac{y_i(t)}{Z(t)} & -\frac{f^2+y_i(t)^2}{f} & \frac{x_i(t)y_i(t)}{f} & x_i(t) \end{bmatrix} \tag{4}$$

where:

- f is the camera's focal length.
- $(x_i(t), y_i(t))$ are the normalized image coordinates related to the camera's intrinsic parameters:
$$x_i(t) = \frac{u_i(t) - c_u}{f\beta}, \quad y_i(t) = \frac{v_i(t) - c_v}{f\beta}$$
Here, $(u_i(t), v_i(t))$ are the pixel coordinates of the i-th feature, (c_u, c_v) denote the principal point coordinates, and β is the scale factor.

2.1.4 Control Law for Conventional IBVS

The conventional IBVS control law aims to minimize the error between current and desired image features by adjusting the camera velocity:

$$V_c(t) = -K\, J_{img}^{+}(t) s_{err}(t) \tag{5}$$

where:

- K is a positive definite gain matrix.
- $J_{img}^{+}(t)$ is the pseudo-inverse of the image Jacobian.
- $s_{err}(t) = s(t) - s_d$ is the feature error vector.

2.2 Adaptive Switch IBVS Controller Design

The adaptive switch IBVS controller introduced by Ghasemi et al. enhances the conventional IBVS by introducing adaptive gains and a switching mechanism that segments the control process into distinct stages. This design addresses the limitations of conventional IBVS, particularly in reducing convergence time and handling camera parameter uncertainties.

2.2.1 Decomposition of Camera Movements

The controller decouples camera movements into rotational and translational components, allowing for specialized control strategies in each phase. This decomposition is facilitated by partitioning the image Jacobian $J_{img}(t)$ into translational $J_t(t)$ and rotational $J_r(t)$ parts:

$$J_{img}(t) = \begin{bmatrix} J_t(t) & J_r(t) \end{bmatrix} \tag{6}$$

where:

$$J_t(t) = \frac{1}{Z(t)} \begin{bmatrix} f & 0 & -x_i(t) \\ 0 & f & -y_i(t) \end{bmatrix}, \quad J_r(t) = \begin{bmatrix} -\frac{x_i(t)y_i(t)}{f} & \frac{f^2+x_i(t)^2}{f} & -y_i(t) \\ -\frac{f^2+y_i(t)^2}{f} & \frac{x_i(t)y_i(t)}{f} & x_i(t) \end{bmatrix} \tag{7}$$

This separation allows the controller to apply distinct control gains to rotational and translational movements, optimizing each for better performance.

2.2.2 Control Stages and Switching Mechanism

The controller operates in three sequential stages, each with tailored control gains:

1. **Stage 1: Pure Rotation**
   - **Objective**: Align the camera's orientation with the desired feature configuration.
   - **Control Action**: Activate only rotational velocity commands $V_{cr}(t)$, setting translational commands $V_{ct}(t) = 0$.
   - **Control Law**:
   $$\dot{s}_i(t) = J_r(t)V_{cr}(t)$$
2. **Stage 2: Pure Translation**
   - **Objective**: Adjust the camera's position to match the desired feature coordinates.

- **Control Action**: Activate only translational velocity commands $V_{ct}(t)$, setting rotational commands $V_{cr}(t) = 0$.
- **Control Law**:
$$\dot{s}_i(t) = J_t(t)V_{ct}(t)$$

3. **Stage 3: Combined Rotation and Translation (Fine-Tuning)**
   - **Objective**: Perform simultaneous rotational and translational adjustments for precise feature alignment.
   - **Control Action**: Activate both $V_{ct}(t)$ and $V_{cr}(t)$.
   - **Control Law**:
$$\dot{s}_i(t) = J_t(t)V_{ct}(t) + J_r(t)V_{cr}(t)$$

The transition between these stages is governed by a switching criterion based on the angle $\alpha(t)$ between the desired and actual feature configurations. Specifically:

- **Switch to Stage 2** when $|\alpha(t)| \geq \alpha_0$.
- **Switch to Stage 3** when $\alpha_1 \leq |\alpha(t)| < \alpha_0$.
- **Remain in Stage 3** when $|\alpha(t)| < \alpha_1$.

2.2.3 Adaptive Control Law and Parameter Estimation

To address uncertainties in camera parameters, the controller employs an adaptive law that estimates these parameters in real-time. The estimation process involves expressing the control equations in a linear parameterization form, facilitating the use of adaptive algorithms to update parameter estimates.

For each feature i during Stage 1 (pure rotation), the image feature velocity equation can be linearized as:

$$\dot{\hat{s}}_i(t) = Y_1(q(t), \dot{q}(t), s(t))\hat{\theta}(t) \tag{8}$$

where:

- $Y_1(q(t), \dot{q}(t), s(t))$ is the regression matrix, independent of camera parameters.
- $\hat{\theta}(t)$ is the vector of estimated parameters, including $\hat{c}_u$, $\hat{c}_v$, $\hat{f}$, and $\hat{\beta}$.

An analogous formulation is applied for Stage 2 (pure translation). The adaptive update laws for the estimated parameters are then defined as:

$$\dot{\hat{\theta}}(t) = K_a^{-1}Y^T K_p \tilde{s}(t) \tag{9}$$

where:

- K_a is a positive definite adaptation gain matrix.
- K_p is a positive definite control gain matrix.
- $\tilde{s}(t) = s(t) - s_d$ is the feature error vector.

During Stage 3, the controller utilizes the estimated camera parameters without further updates, relying on the previously obtained estimates for precise control.

2.2.4 Torque Control and Robot Dynamics

In the original paper, Ghasemi et al. integrate the robot's dynamic model into the control strategy to achieve more refined and stable movements. The dynamic equation governing the robot manipulator is expressed as:

$$M(q(t))\ddot{q}(t) + C(q(t), \dot{q}(t))\dot{q}(t) + G(q(t)) = \tau \tag{10}$$

where:

- $M(q(t))$ is the inertia matrix.
- $C(q(t), \dot{q}(t))$ represents Coriolis and centrifugal forces.
- $G(q(t))$ accounts for gravitational forces.
- τ is the joint torque vector.

The control strategy involves generating joint torques τ based on the desired camera velocities $V_c(t)$, utilizing the robot's Jacobian matrix $J_R(t)$:

$$V_c(t) = J_R(t)\dot{q}(t) \tag{11}$$

where $J_R(t)$ is the 6×6 geometric Jacobian matrix, decomposed into translational $J_{Rt}(t)$ and rotational $J_{Rr}(t)$ components:

$$J_R(t) = \begin{bmatrix} J_{Rt}(t) \\ J_{Rr}(t) \end{bmatrix} \tag{12}$$

This decomposition aligns with the image Jacobian's structure, facilitating the decoupled control of translational and rotational movements.

2.2.5 Control Law Based on Stages

Equations (13), (14) and (15) delineate the control law for generating joint torque commands τ based on the current control stage. The control law is expressed as:

$$\tau_{s1} = G(q(t)) - K_{v1}\dot{q}(t) - \left(\hat{J}_r(t) J_{Rr}(q(t))^T K_{p1}\tilde{s}(t)\right), \quad |\alpha| \geq \alpha_0 \tag{13}$$

$$\tau_{s2} = G(q(t)) - K_{v2}\dot{q}(t) - \left(\hat{J}_t(t) J_{Rt}(q(t))^T K_{p2}\tilde{s}(t)\right), \quad \alpha_1 \leq |\alpha| < \alpha_0 \tag{14}$$

$$\tau_{s3} = G(q(t)) - K_{v3}\dot{q}(t) - \left(\hat{J}_{img}(t) J_R(q(t))^T K_{p3}\tilde{s}(t)\right), \quad \text{otherwise} \tag{15}$$

where:

- $\tau_{s1}, \tau_{s2}, \tau_{s3}$ are the torque commands for Stages 1, 2, and 3, respectively.
- $G(q(t))$ represents the gravitational forces.

- K_{v1}, K_{v2}, K_{v3} are positive definite gain matrices for velocity damping in each stage.
- K_{p1}, K_{p2}, K_{p3} are positive definite proportional gain matrices for feature error correction in each stage.
- $\hat{J}_r(t)$ and $\hat{J}_t(t)$ are the estimated rotational and translational parts of the image Jacobian, respectively.
- $J_{Rr}(q(t))$ and $J_{Rt}(q(t))$ are the rotational and translational parts of the robot Jacobian.
- $\tilde{s}(t) = s(t) - s_d$ is the feature error vector.
- α is the angle between desired and actual feature configurations, dictating stage transitions.

This equation ensures that, depending on the current stage determined by α, the controller applies appropriate torque commands that consider both the gravitational effects and the desired feature corrections, modulated by stage-specific gains.

2.2.6 Update Law for Uncertain Camera Parameters

Equations (16), (17) and (18) define the adaptive update law for estimating uncertain camera parameters $\hat{\theta}(t)$. The update law is formulated as:

$$\dot{\hat{\theta}}(t) = K_a^{-1} Y_1^T K_{p1} \tilde{s}(t), \quad |\alpha| \geq \alpha_0 \tag{16}$$

$$\dot{\hat{\theta}}(t) = K_a^{-1} Y_2^T K_{p2} \tilde{s}(t), \quad \alpha_1 \leq |\alpha| < \alpha_0 \tag{17}$$

$$\dot{\hat{\theta}}(t) = 0, \quad \text{otherwise} \tag{18}$$

where:

- $\hat{\theta}(t)$ is the vector of estimated camera parameters.
- K_a is a positive definite adaptation gain matrix.
- Y_1 and Y_2 are regression matrices corresponding to Stages 1 and 2, respectively.
- K_{p1} and K_{p2} are positive definite proportional gain matrices for Stages 1 and 2.
- $\tilde{s}(t) = s(t) - s_d$ is the feature error vector.

This adaptive law facilitates the real-time estimation of camera parameters by updating $\hat{\theta}(t)$ based on the current feature errors and the regression matrices, which encapsulate the relationship between camera movements and feature errors. During Stage 3, where both rotational and translational movements are active, the camera parameters are no longer updated, relying instead on the estimates obtained from the preceding stages.

2.3 Implementation Adaptations

In translating the theoretical framework to a MATLAB simulation, several practical considerations were addressed. While the original paper integrates dynamic modeling and torque control, my implementation focuses on a kinematic approach due to the absence of detailed dynamic models for the Universal UR5 robot.

2.3.1 Simulation Environment and Robot Model

Utilizing MATLAB's `loadrobot` function, the Universal UR5 robot is simulated with its kinematic configuration. Initial joint angles are set to zero, and the robot's movement is governed by joint velocities rather than torque commands, simplifying the control strategy within the simulated environment.

2.3.2 Camera Model and Feature Projection

A simulated monocular camera is defined with intrinsic parameters including focal length (f), principal point coordinates (c_u, c_v), and a scale factor (β). Object features are projected into the image plane using these intrinsic parameters, assuming a constant depth Z_{des} for all features to simplify the image Jacobian calculations.

2.3.3 Control Gains and Switching Thresholds

Stage-specific control gains ($K_{\text{stage1}}, K_{\text{stage2}}, K_{\text{stage3}}$) are predefined to modulate the responsiveness of the controller during different movement phases. Switching thresholds (α_0 and α_1) are set based on the initial angle between desired and actual feature configurations, dictating transitions between control stages.

2.3.4 Control Signal Generation and Integration

Based on the current control stage, the image Jacobian $J_{img}(t)$ is constructed by selectively activating translational and/or rotational components. The desired camera velocity $V_c(t)$ is computed using a velocity-based control law:

$$V_c(t) = -K_{\text{stage}} \cdot J_{img}^{+}(t) \cdot s_{err}(t) \tag{19}$$

where K_{stage} varies depending on the active control stage. This velocity is then integrated over discrete time steps to update the camera's pose (position and orientation) within the simulation.

2.3.5 Robot Joint Velocity Update

The corresponding joint velocities $\dot{q}(t)$ are derived from the desired camera velocities using the robot's geometric Jacobian $J_R(t)$:

$$\dot{q}(t) = J_R^{+}(t) \cdot V_c(t) \tag{20}$$

where $J_R^{+}(t)$ is the pseudo-inverse of the robot Jacobian. This computation ensures that the joint velocities are adjusted to achieve the desired end-effector movement corresponding to the IBVS control commands.

2.3.6 Logging and Visualization

Throughout the simulation, data on feature positions, errors, camera poses, and control stages are logged for post-simulation analysis. Various plots are generated to visualize the trajectories of image features, the evolution of feature errors, the camera's pose over time, and the switching between control stages.

2.3.7 Omissions Due to Implementation Constraints

Given the absence of dynamic modeling for the Universal UR5 robot and the focus on a simulated environment, the following aspects from the original paper were omitted:

1. **Torque Control**: Instead of generating torque commands, joint velocities are directly controlled to simulate the robot's movement. This simplification bypasses the need for detailed dynamic equations and torque-based control laws.

2. **Adaptive Parameter Estimation**: The regression matrix calculation and the adaptive update laws for uncertain camera parameters ($\hat{Q}(t)$ and $\hat{Q}_i(t)$) were not implemented. Consequently, the controller relies on predefined camera parameters rather than adapting to parameter uncertainties in real-time.

Despite these omissions, the core objective of enhancing IBVS performance through stage-based control remains achieved within the simulated framework.

2.4 Mathematical Modeling in Implementation

The core mathematical framework from the original paper is retained, albeit adapted for a simulated setup. Below are the key equations and their adaptations:

2.4.1 Projected Image Features

The projection of world coordinates to image coordinates is modeled using the camera intrinsic matrix K and the camera pose defined by rotation R and translation t:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \cdot \frac{R^T(P_{\text{world}} - t)}{Z_{\text{world}}} \tag{21}$$

where P_{world} represents the world coordinates of object features. In the simulation, a function `projectPoints` handles this projection, assuming a constant depth Z_{des}.

2.4.2 Control Law Implementation

Based on the current control stage, the image Jacobian $J_{img}(t)$ is constructed by selectively activating translational and/or rotational components:

- **Stage 1 (Pure Rotation)**:

$$J_{img}^{(1)}(t) = \begin{bmatrix} 0 & 0 & 0 & J_r(t) \end{bmatrix} \tag{22}$$

- **Stage 2 (Pure Translation)**:

$$J_{img}^{(2)}(t) = \begin{bmatrix} J_t(t) & 0 & 0 & 0 \end{bmatrix} \tag{23}$$

- **Stage 3 (Combined)**:

$$J_{img}^{(3)}(t) = \begin{bmatrix} J_t(t) & J_r(t) \end{bmatrix} \tag{24}$$

The control velocity $V_c(t)$ is then computed as:

$$V_c(t) = -K_{\text{stage}} \cdot J_{img}^{+}(t) \cdot s_{err}(t) \tag{25}$$

where K_{stage} and $J_{\text{img}}^{+}(t)$ vary based on the current control stage.

2.4.3 Switching Criterion

The angle $\alpha(t)$ between the desired and actual feature configurations serves as the switching criterion:

$$\alpha(t) = \arccos\left(\frac{\vec{v}_{\text{act}} \cdot \vec{v}_{\text{des}}}{\|\vec{v}_{\text{act}}\|\|\vec{v}_{\text{des}}\|}\right) \tag{26}$$

where $\vec{v}_{\text{act}}$ and $\vec{v}_{\text{des}}$ are vectors representing the actual and desired feature configurations, respectively. In the simulation, $\alpha(t)$ is computed based on the centroids of the actual and desired feature positions, providing an intuitive measure for stage transitions.

2.4.4 Stage-Based Jacobian Modification

Adaptation in My Implementation Since torque control is not implemented in my setup, the essence of stage-based control is preserved by manipulating the robot's Jacobian matrix based on the current control stage in the same way explained in the paper. This approach allows for selective control of either rotational or translational movements, or both, by modifying the Jacobian used to compute joint velocities.

In my MATLAB implementation, the robot's Jacobian $J_R(t)$ is adjusted according to the active control stage as follows:

- **Stage 1: Pure Rotation**
  - Only the rotational components of the Jacobian are active.
  - The translational components are nullified by setting the corresponding columns to zero.
  - This ensures that only rotational velocities are commanded, aligning the camera's orientation with the desired configuration.

- **Stage 2: Pure Translation**
  - Only the translational components of the Jacobian are active.
  - The rotational components are nullified by setting the corresponding columns to zero.
  - This configuration commands only translational velocities, adjusting the camera's position to match the desired feature coordinates.
- **Stage 3: Combined Rotation and Translation**
  - Both translational and rotational components of the Jacobian are active.
  - This allows for simultaneous adjustment of both the camera's position and orientation for fine-tuning feature alignment.

Mathematical Representation The modified Jacobian $J_{R_{\text{stage}}}(t)$ for each stage is constructed by selectively retaining the relevant submatrices:

$$J_{R_{\text{stage1}}}(t) = \begin{bmatrix} \mathbf{0}_{3\times n} & J_{Rr}(t) \end{bmatrix}, \quad |\alpha| \geq \alpha_0 \tag{27}$$

$$J_{R_{\text{stage2}}}(t) = \begin{bmatrix} J_{Rt}(t) & \mathbf{0}_{3\times m} \end{bmatrix}, \quad \alpha_1 \leq |\alpha| < \alpha_0 \tag{28}$$

$$J_{R_{\text{stage3}}}(t) = J_R(t), \quad \text{otherwise} \tag{29}$$

where:

- $\mathbf{0}_{a\times b}$ denotes an $a \times b$ matrix of zeros.
- $J_{Rt}(t)$ and $J_{Rr}(t)$ are the translational and rotational parts of the robot Jacobian, respectively.
- n and m correspond to the number of translational and rotational degrees of freedom being nullified.

2.4.5 Control Law Implementation

Given the desired camera velocity $V_c(t)$, the corresponding joint velocities $\dot{q}(t)$ are computed using the stage-specific Jacobian $J_{R_{\text{stage}}}(t)$, the joint velocity command $\dot{q}(t)$ is computed as:

$$\dot{q}(t) = J^{+}_{R_{\text{stage}}}(t) \cdot V_c(t) \tag{30}$$

where $J^{+}_{R_{\text{stage}}}(t)$ is the pseudo-inverse of the modified Jacobian for the current stage, and $V_c(t)$ is the desired camera velocity computed based on the IBVS control law.

This approach effectively emulates the selective torque commands of the original Equation (13) by constraining the robot's movements to either rotational, translational, or combined actions based on the current stage.

3 Stability Proof

In this section, we present the stability proof of the proposed adaptive switch IBVS controller as outlined in the original paper by Ghasemi et al. [1]. The proof demonstrates that the feature error vector $\tilde{s}(t) = s(t) - s_d$ asymptotically converges to zero under the proposed control law and adaptive update mechanisms.

3.1 Preliminaries

3.1.1 Property 1

Before delving into the stability proof, it is essential to establish a fundamental property related to the robot's dynamic equation.

Property 1: The matrix $\dot{M}(q(t)) - 2C(q(t), \dot{q}(t))$ is skew-symmetric. Mathematically, this is expressed as:

$$\delta^T \left[\dot{M}(q(t)) - 2C(q(t), \dot{q}(t)) \right] \delta = 0, \quad \forall \delta \in \mathbb{R}^n. \tag{31}$$

This property is pivotal in simplifying the derivative of the Lyapunov function and ensuring the cancellation of certain terms during the stability analysis.

3.2 Theorem 1

Theorem 1: Consider a 6 DOF robot with an eye-in-hand camera installed at the end-effector, governed by the dynamic equation (2.2.4). The proposed adaptive switch controller (13)-(15) with the update law (16)-(18) ensures that the feature error $\tilde{s}(t)$ asymptotically converges to zero, i.e.,

$$\lim_{t \to \infty} \tilde{s}(t) = 0.$$

3.3 Proof of Theorem 1

The proof of Theorem 1 is conducted by analyzing each control stage individually—Stage 1 (Pure Rotation), Stage 2 (Pure Translation), and Stage 3 (Combined Rotation and Translation). The Lyapunov stability method is employed to demonstrate that the feature error converges to zero.

3.3.1 Stage 1: Pure Rotation

Lyapunov Function Candidate For Stage 1, where pure rotation is active, we define the following Lyapunov function candidate:

$$V(t) = \frac{1}{2} \left[\dot{q}(t)^T M(q(t)) \dot{q}(t) + \tilde{s}(t)^T K_{p1} \tilde{s}(t) + \tilde{\theta}(t)^T K_a \tilde{\theta}(t) \right], \tag{32}$$

where:

- $\dot{q}(t)$ is the joint velocity vector.
- $M(q(t))$ is the inertia matrix.
- K_{p1} is the proportional gain matrix for Stage 1.

- K_a is the adaptation gain matrix.
- $\tilde{\theta}(t) = \theta(t) - \hat{\theta}(t)$ represents the parameter estimation error.

Derivative of the Lyapunov Function Differentiating $V(t)$ with respect to time yields:

$$\dot{V}(t) = \dot{q}(t)^T M(q(t))\ddot{q}(t) + \frac{1}{2}\dot{q}(t)^T \dot{M}(q(t))\dot{q}(t) + \tilde{s}(t)^T K_{p1}\dot{s}(t) - \tilde{\theta}(t)^T K_a \dot{\hat{\theta}}(t). \tag{33}$$

Substitution Using Dynamic Equation Substituting the robot's dynamic equation (2.2.4) into the derivative, we obtain:

$$\begin{aligned}\dot{V}(t) = \dot{q}(t)^T(\tau(t) - G(q(t))) + \frac{1}{2}\dot{q}(t)^T \left[\dot{M}(q(t)) - 2C(q(t), \dot{q}(t))\right]\dot{q}(t) \\ + \tilde{s}(t)^T K_p \dot{s}(t) - \tilde{\theta}(t)^T K_a \dot{\hat{\theta}}(t).\end{aligned} \tag{34}$$

Application of Property 1 Using Property 1, the skew-symmetry of $\dot{M}(q(t)) - 2C(q(t), \dot{q}(t))$ implies that:

$$\frac{1}{2}\dot{q}(t)^T \left[\dot{M}(q(t))\dot{q}(t) - 2C(q(t), \dot{q}(t))\dot{q}(t)\right] = 0.$$

Therefore, Equation (34) simplifies to:

$$\dot{V}(t) = \dot{q}(t)^T \tau - \dot{q}(t)^T G(q(t)) + \tilde{s}(t)^T K_{p1}\dot{s}(t) - \tilde{\theta}(t)^T K_a \dot{\hat{\theta}}(t). \tag{35}$$

Substitution of Control Law Substituting the control torque τ from the first stage of the adaptive control law (13):

$$\tau_{s1} = G(q(t)) - K_{v1}\dot{q}(t) - \left(\hat{J}_r(t) J_{Rr}(q(t))^T K_{p1}\tilde{s}(t)\right),$$

we have:

$$\dot{V}(t) = \dot{q}(t)^T \left(-K_{v1}\dot{q}(t) - \hat{J}_r(t) J_{Rr}(q(t))^T K_{p1}\tilde{s}(t)\right) - \dot{q}(t)^T G(q(t)) + \tilde{s}(t)^T K_{p1}\dot{s}(t) - \tilde{\theta}(t)^T K_a \dot{\hat{\theta}}(t). \tag{36}$$

Utilization of Update Law Using the update law from the first stage (16):

$$\dot{\hat{\theta}}(t) = K_a^{-1} Y_1^T K_{p1}\tilde{s}(t),$$

and recognizing that $\dot{s}(t) = \hat{J}_r(t)\dot{q}(t)$, we can further simplify Equation (36):

$$\begin{aligned}\dot{V}(t) &= \dot{q}(t)^T(-K_{v1}\dot{q}(t)) - \dot{q}(t)^T \hat{J}_r(t) J_{Rr}(q(t))^T K_{p1}\tilde{s}(t) + \tilde{s}(t)^T K_{p1}\hat{J}_r(t)\dot{q}(t) \\ &\quad - \tilde{\theta}(t)^T K_a K_a^{-1} Y_1^T K_{p1}\tilde{s}(t) \\ &= -\dot{q}(t)^T K_{v1}\dot{q}(t) - \tilde{\theta}(t)^T Y_1^T K_{p1}\tilde{s}(t) + \tilde{s}(t)^T K_{p1}\hat{J}_r(t)\dot{q}(t) \\ &\quad - \tilde{\theta}(t)^T Y_1^T K_{p1}\tilde{s}(t).\end{aligned} \tag{37}$$

Final Simplification Notice that the terms involving $\tilde{s}(t)$ and $\tilde{\theta}(t)$ cancel out:

$$-\tilde{\theta}(t)^T Y_1^T K_{p1}\tilde{s}(t) + \tilde{s}(t)^T K_{p1}\hat{J}_r(t)\dot{q}(t) - \tilde{\theta}(t)^T Y_1^T K_{p1}\tilde{s}(t) = 0.$$

Thus, the derivative simplifies to:

$$\dot{V}(t) = -\dot{q}(t)^T K_{v1}\dot{q}(t) \leq 0. \tag{38}$$

Conclusion for Stage 1 Since $\dot{V}(t) \leq 0$, the Lyapunov function is non-increasing. According to Barbălat's Lemma [2], if $V(t)$ is lower bounded and $\dot{V}(t)$ is uniformly continuous and $\lim_{t\to\infty}\dot{V}(t) = 0$, then $\tilde{s}(t)$ asymptotically converges to zero. Therefore, the stability of the adaptive controller in Stage 1 is established.

3.3.2 Stage 2: Pure Translation

Lyapunov Function Candidate For Stage 2, where pure translation is active, we maintain the same Lyapunov function candidate as in Stage 1:

$$V(t) = \frac{1}{2}\left[\dot{q}(t)^T M(q(t))\dot{q}(t) + \tilde{s}(t)^T K_{p1}\tilde{s}(t) + \tilde{\theta}(t)^T K_a\tilde{\theta}(t)\right]. \tag{39}$$

Derivative of the Lyapunov Function Differentiating $V(t)$ with respect to time:

$$\dot{V}(t) = \dot{q}(t)^T M(q(t))\ddot{q}(t) + \frac{1}{2}\dot{q}(t)^T \dot{M}(q(t))\dot{q}(t) + \tilde{s}(t)^T K_{p1}\dot{s}(t) - \tilde{\theta}(t)^T K_a\dot{\hat{\theta}}(t). \tag{40}$$

Substitution Using Dynamic Equation Substituting the dynamic equation (2.2.4) and applying Property 1:

$$\dot{V}(t) = \dot{q}(t)^T\left(\tau - G(q(t))\right) + \frac{1}{2}\dot{q}(t)^T\left[\dot{M}(q(t))\dot{q}(t) - 2C(q(t),\dot{q}(t))\dot{q}(t)\right] + \tilde{s}(t)^T K_{p1}\dot{s}(t) - \tilde{\theta}(t)^T K_a\dot{\hat{\theta}}(t). \tag{41}$$

Applying Property 1 simplifies this to:

$$\dot{V}(t) = \dot{q}(t)^T\tau - \dot{q}(t)^T G(q(t)) + \tilde{s}(t)^T K_{p1}\dot{s}(t) - \tilde{\theta}(t)^T K_a\dot{\hat{\theta}}(t). \tag{42}$$

Substitution of Control Law Substituting the control torque τ from the second stage of the adaptive control law (14):

$$\tau_{s2} = G(q(t)) - K_{v2}\dot{q}(t) - \left(\hat{J}_t(t)J_{Rt}(q(t))^T K_{p2}\tilde{s}(t)\right),$$

we obtain:

$$\dot{V}(t) = \dot{q}(t)^T\left(-K_{v2}\dot{q}(t) - \hat{J}_t(t)J_{Rt}(q(t))^T K_{p2}\tilde{s}(t)\right) - \dot{q}(t)^T G(q(t)) + \tilde{s}(t)^T K_{p1}\dot{s}(t) - \tilde{\theta}(t)^T K_a\dot{\hat{\theta}}(t). \tag{43}$$

Utilization of Update Law Using the update law from the second stage (17):

$$\dot{\hat{\theta}}(t) = K_a^{-1} Y_2^T K_{p2} \tilde{s}(t),$$

and recognizing that $\dot{s}(t) = \hat{J}_t(t)\dot{q}(t)$, we can further simplify Equation (43):

$$\begin{aligned}
\dot{V}(t) &= \dot{q}(t)^T(-K_{v2}\dot{q}(t)) - \dot{q}(t)^T \hat{J}_t(t) J_{Rt}(q(t))^T K_{p2}\tilde{s}(t) + \tilde{s}(t)^T K_{p1}\hat{J}_t(t)\dot{q}(t) \\
&\quad - \tilde{\theta}(t)^T K_a K_a^{-1} Y_2^T K_{p2}\tilde{s}(t) \\
&= -\dot{q}(t)^T K_{v2}\dot{q}(t) - \tilde{\theta}(t)^T Y_2^T K_{p2}\tilde{s}(t) + \tilde{s}(t)^T K_{p1}\hat{J}_t(t)\dot{q}(t) \\
&\quad - \tilde{\theta}(t)^T Y_2^T K_{p2}\tilde{s}(t).
\end{aligned} \tag{44}$$

Assumption on Proportional Gains It is assumed that:

$$K_{p1} = K_{p2} + K_\alpha,$$

where K_α is a positive definite matrix.

Further Simplification Substituting the above assumption into Equation (44), we obtain:

$$\begin{aligned}
\dot{V}(t) &= -\dot{q}(t)^T K_{v2}\dot{q}(t) - \tilde{\theta}(t)^T Y_2^T K_{p2}\tilde{s}(t) + \tilde{s}(t)^T K_{p1}\hat{J}_t(t)\dot{q}(t) - \tilde{\theta}(t)^T Y_2^T K_{p2}\tilde{s}(t) \\
&= -\dot{q}(t)^T K_{v2}\dot{q}(t) + \tilde{s}(t)^T K_\alpha \tilde{s}(t).
\end{aligned} \tag{45}$$

This equation can be rearranged as:

$$\dot{V}(t) = -2V(t) + \epsilon_1, \tag{46}$$

where:

$$\begin{aligned}
\epsilon_1 &= \dot{q}(t)^T(-K_{v2})\dot{q}(t) + \dot{s}(t)^T K_\alpha \tilde{s}(t) + 2V(t) \\
&= \dot{q}(t)^T(-K_{v2})\dot{q}(t) + \dot{s}(t)^T K_\alpha \tilde{s}(t) + \dot{q}(t)^T M(q(t))\dot{q}(t) + \tilde{s}(t)^T K_{p1}\tilde{s}(t) + \tilde{\theta}(t)^T K_a \tilde{\theta}(t) \\
&= \dot{q}(t)^T(M(q(t)) - K_{v2})\dot{q}(t) + \dot{s}(t)^T K_\alpha \tilde{s}(t) + \tilde{s}(t)^T K_{p1}\tilde{s}(t) + \tilde{\theta}(t)^T K_a \tilde{\theta}(t).
\end{aligned} \tag{47}$$

Since K_{v2} is chosen such that $M(q(t)) - K_{v2} \geq 0$, and all other terms are positive definite, we can infer that:

$$\epsilon_1 \leq \bar{\epsilon}_1, \tag{48}$$

where $\bar{\epsilon}_1$ is a positive constant.

Conclusion for Stage 2 From Equation (46), we have:

$$\dot{V}(t) \leq -2V(t) + \bar{\epsilon}_1.$$

Applying the boundedness theorem, it follows that the system's stability in Stage 2 is ensured, and the feature error $\tilde{s}(t)$ asymptotically converges to zero.

3.3.3 Stage 3: Combined Rotation and Translation

Lyapunov Function Candidate For Stage 3, where both rotation and translation are active, the same Lyapunov function candidate is utilized:

$$V(t) = \frac{1}{2}\left[\dot{q}(t)^T M(q(t))\dot{q}(t) + \tilde{s}(t)^T K_{p1}\tilde{s}(t) + \tilde{\theta}(t)^T K_a \tilde{\theta}(t)\right]. \tag{49}$$

Derivative of the Lyapunov Function Differentiating $V(t)$ with respect to time:

$$\dot{V}(t) = \dot{q}(t)^T M(q(t))\ddot{q}(t) + \frac{1}{2}\dot{q}(t)^T \dot{M}(q(t))\dot{q}(t) + \tilde{s}(t)^T K_{p1}\dot{s}(t) - \tilde{\theta}(t)^T K_a \dot{\hat{\theta}}(t). \tag{50}$$

Substitution Using Dynamic Equation Substituting the dynamic equation (2.2.4) and applying Property 1:

$$\dot{V}(t) = \dot{q}(t)^T\left(\tau - G(q(t))\right) + \frac{1}{2}\dot{q}(t)^T\left[\dot{M}(q(t))\dot{q}(t) - 2C(q(t),\dot{q}(t))\dot{q}(t)\right] + \tilde{s}(t)^T K_{p1}\dot{s}(t) - \tilde{\theta}(t)^T K_a \dot{\hat{\theta}}(t). \tag{51}$$

Applying Property 1 simplifies this to:

$$\dot{V}(t) = \dot{q}(t)^T\tau - \dot{q}(t)^T G(q(t)) + \tilde{s}(t)^T K_{p1}\dot{s}(t) - \tilde{\theta}(t)^T K_a \dot{\hat{\theta}}(t). \tag{52}$$

Substitution of Control Law Substituting the control torque τ from the third stage of the adaptive control law (15):

$$\tau_{s3} = G(q(t)) - K_{v3}\dot{q}(t) - \left(\hat{J}_{img}(t) J_R(q(t))^T K_{p3}\tilde{s}(t)\right),$$

we obtain:

$$\dot{V}(t) = \dot{q}(t)^T\left(-K_{v3}\dot{q}(t) - \hat{J}_{img}(t) J_R(q(t))^T K_{p3}\tilde{s}(t)\right) - \dot{q}(t)^T G(q(t)) + \tilde{s}(t)^T K_{p1}\dot{s}(t) - \tilde{\theta}(t)^T K_a \dot{\hat{\theta}}(t). \tag{53}$$

Utilization of Update Law Using the update law from the third stage (18):

$$\dot{\hat{\theta}}(t) = K_a^{-1} Y_3^T K_{p3}\tilde{s}(t),$$

and recognizing that $\dot{s}(t) = \hat{J}_{img}(t)\dot{q}(t)$, we can further simplify Equation (53):

$$\begin{aligned}\dot{V}(t) &= \dot{q}(t)^T(-K_{v3}\dot{q}(t)) - \dot{q}(t)^T \hat{J}_{img}(t) J_R(q(t))^T K_{p3}\tilde{s}(t) + \tilde{s}(t)^T K_{p1}\hat{J}_{img}(t)\dot{q}(t) \\ &\quad - \tilde{\theta}(t)^T K_a K_a^{-1} Y_3^T K_{p3}\tilde{s}(t) \\ &= -\dot{q}(t)^T K_{v3}\dot{q}(t) - \tilde{\theta}(t)^T Y_3^T K_{p3}\tilde{s}(t) + \tilde{s}(t)^T K_{p1}\hat{J}_{img}(t)\dot{q}(t) \\ &\quad - \tilde{\theta}(t)^T Y_3^T K_{p3}\tilde{s}(t).\end{aligned} \tag{54}$$

Assumption on Proportional Gains It is assumed that:

$$K_{p1} = K_{p3} + K_\beta,$$

where K_β is a positive definite matrix.

Further Simplification Substituting the above assumption into Equation (54), we obtain:

$$\dot{V}(t) = -\dot{q}(t)^T K_{v3}\dot{q}(t) - \tilde{\theta}(t)^T Y_3^T K_{p3}\tilde{s}(t) + \tilde{s}(t)^T K_\beta \tilde{s}(t). \tag{55}$$

This equation can be rearranged as:

$$\dot{V}(t) = -2V(t) + \epsilon_2, \tag{56}$$

where:

$$\begin{aligned}
\epsilon_2 &= \dot{q}(t)^T(-K_{v3})\dot{q}(t) + \tilde{\theta}(t)^T Y_3^T K_{p3}\tilde{s}(t) + \dot{s}(t)^T K_\beta \tilde{s}(t) + 2V(t) \\
&= \dot{q}(t)^T(-K_{v3})\dot{q}(t) + \tilde{\theta}(t)^T Y_3^T K_{p3}\tilde{s}(t) + \dot{s}(t)^T K_\beta \tilde{s}(t) + \dot{q}(t)^T M(q(t))\dot{q}(t) \\
&+ \tilde{s}(t)^T K_{p1}\tilde{s}(t) + \tilde{\theta}(t)^T K_a \tilde{\theta}(t) \\
&= \dot{q}(t)^T(M(q(t)) - K_{v3})\dot{q}(t) + \dot{s}(t)^T K_\beta \tilde{s}(t) + \tilde{s}(t)^T K_{p1}\tilde{s}(t) + \tilde{\theta}(t)^T K_a \tilde{\theta}(t).
\end{aligned} \tag{57}$$

Boundedness of ϵ_2 Since K_{v3} is chosen such that $M(q(t)) - K_{v3} \geq 0$, and all other terms are positive definite, we can infer that:

$$\epsilon_2 \leq \bar{\epsilon}_2,$$

where $\bar{\epsilon}_2$ is a positive constant.

Conclusion for Stage 3 From Equation (56), we have:

$$\dot{V}(t) \leq -2V(t) + \bar{\epsilon}_2.$$

Applying the boundedness theorem, it follows that the system's stability in Stage 3 is ensured, and the feature error $\tilde{s}(t)$ asymptotically converges to zero.

3.4 Overall Stability Conclusion

By analyzing each control stage independently and demonstrating that the derivative of the Lyapunov function $\dot{V}(t)$ is non-positive and bounded, we establish that the adaptive switch IBVS controller guarantees the asymptotic convergence of the feature error $\tilde{s}(t)$ to zero across all stages. This ensures robust and stable performance of the controller in guiding the robot's end-effector towards the desired feature configuration.

4 Implementation Results

The results presented in this section demonstrate the comparative performance of the Traditional Image-Based Visual Servoing (IBVS) and the Adaptive Switch IBVS methods. The primary focus is to showcase the Adaptive Switch IBVS's ability to achieve successful convergence across a wider range of orientation mismatches, addressing inherent limitations observed in conventional IBVS approaches. Both methodologies are evaluated under identical initial and desired configurations, with varying degrees of orientation mismatches to assess their robustness and efficiency.

4.1 Parameters and Test Cases

4.1.1 Initial and Desired Configurations

The experiments were conducted using a Universal Robots UR5 model within a simulated MATLAB environment. The initial configuration of the camera was set with the following parameters:

- **Position**: $t_{\text{init}} = [0, 0, 0.7]^T$ meters.
- **Orientation**: Defined by successive rotations of 10° about the X-axis, 30° about the Y-axis, and 60° about the Z-axis.

The desired configuration was established as:

- **Position**: $t_{\text{des}} = [0.05, 0, 0.9]^T$ meters.
- **Orientation**: Identity matrix (no rotation), i.e., $R_{\text{des}} = I_{3\times3}$.

4.1.2 Switching Parameters

For the Adaptive Switch IBVS, the switching thresholds were set as:

- $\alpha_0 = 15°$: Determines the transition from Stage 1 (Pure Rotation) to Stage 2 (Pure Translation).
- $\alpha_1 = 8°$: Governs the switch from Stage 2 to Stage 3 (Combined Rotation and Translation).

These parameters are pivotal in dictating the control stage transitions based on the current orientation error.

4.1.3 Control Gains

The control gains assigned for each stage are as follows:

- **Traditional IBVS**: Fixed gain $\lambda = 4.5$.
- **Adaptive Switch IBVS**:
  - $K_{\text{stage1}} = 2$ (Pure Rotation)
  - $K_{\text{stage2}} = 4$ (Pure Translation)
  - $K_{\text{stage3}} = 6$ (Combined Rotation and Translation)

4.1.4 Convergence Criteria

The convergence of each method was determined based on the norm of the feature error vector $||\tilde{s}(t)||$. A convergence was declared when $||\tilde{s}(t)|| < 10^{-3}$ within a maximum of 1000 iterations. Failure to meet this criterion within the iteration limit was considered a non-convergence.

4.1.5 Test Cases

The following test cases were conducted to evaluate the performance of both IBVS methods:

Traditional IBVS:

1. Case 1: Orientation Mismatch $[30°, 30°, 30°]$
   Result: Converged at iteration 96.

2. Case 2: Orientation Mismatch $[10°, 30°, 60°]$
   Result: Converged at iteration 95.

3. Case 3: Orientation Mismatch $[10°, 15°, 200°]$
   Result: Failed to converge within 1000 iterations.

Adaptive Switch IBVS:

1. Case 1: Orientation Mismatch $[30°, 30°, 30°]$
   Result: Converged at iteration 70.

2. Case 2: Orientation Mismatch $[10°, 30°, 60°]$
   Result: Converged at iteration 63.

3. Case 3: Orientation Mismatch $[10°, 15°, 200°]$
   Result: Converged at iteration 59.

4. Extreme Case: Orientation Mismatch $[200°, 110°, -130°]$
   Control Gains: $K = [2, 4, 6]$
   Result: Converged at iteration 108.

These cases illustrate the Adaptive Switch IBVS's superior capability in handling larger orientation mismatches and achieving faster convergence compared to the Traditional IBVS.

4.2 Traditional IBVS Implementation Results

This subsection presents the implementation results of the Traditional IBVS method. Seven plots are generated for each test case, providing insights into the feature trajectories, error norms, camera poses, camera trajectories, camera velocities, joint angles, and joint velocities over iterations.

4.2.1 Case 1: Orientation Mismatch [30°, 30°, 30°]

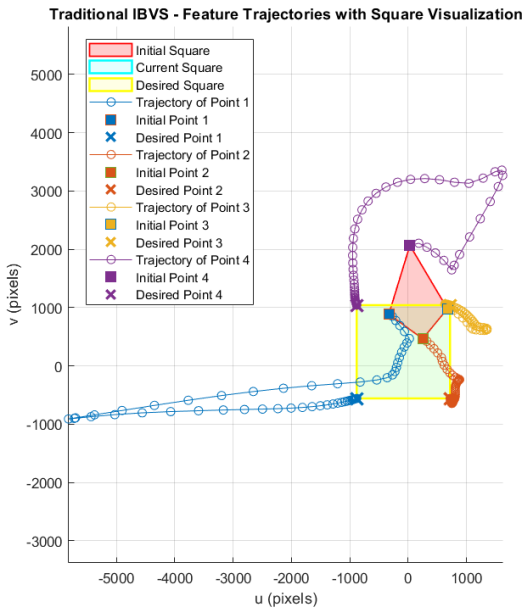

Figure 1: Traditional IBVS - Case 1: Feature Trajectories

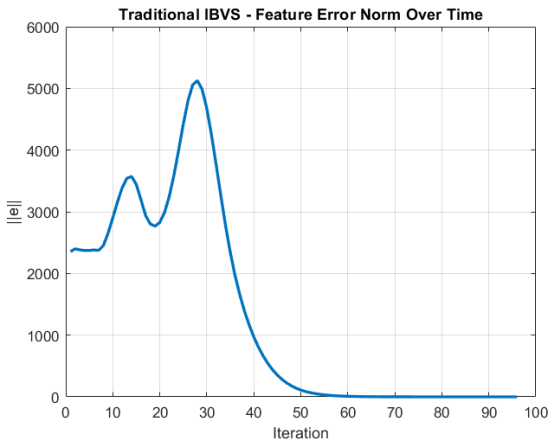

Figure 2: Traditional IBVS - Case 1: Feature Error Norm Over Iterations

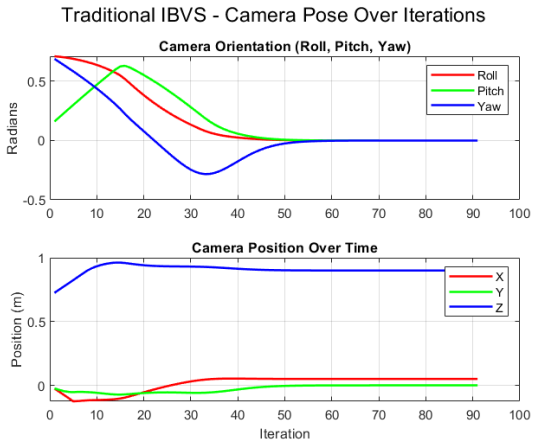

Figure 3: Traditional IBVS - Case 1: Camera Pose Over Iterations

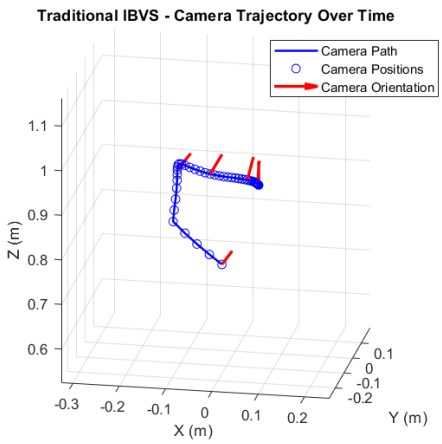

Figure 4: Traditional IBVS - Case 1: Camera Trajectory

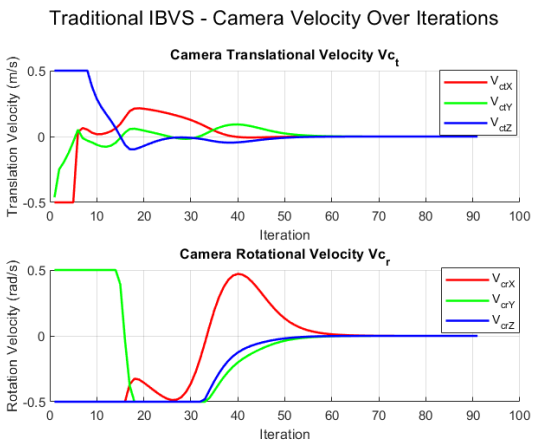

Figure 5: Traditional IBVS - Case 1: Camera Velocity Over Iterations

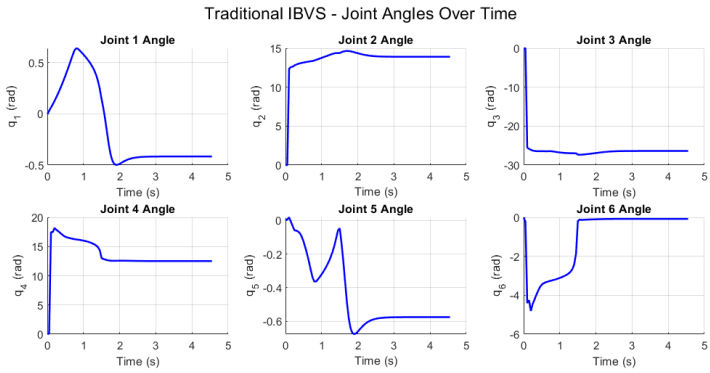

Figure 6: Traditional IBVS - Case 1: Joint Angles Over Time

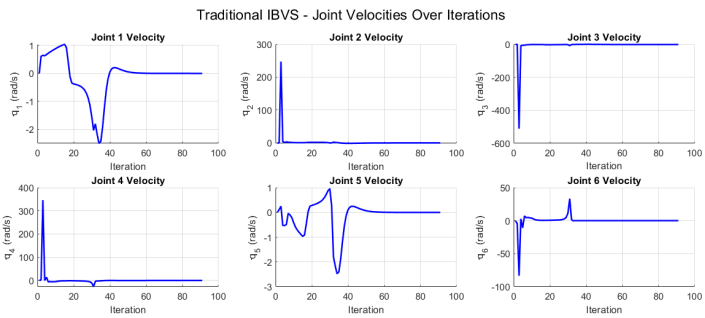

Figure 7: Traditional IBVS - Case 1: Joint Velocities Over Iterations

Explanation of Plots

- **Feature Trajectories (Figure 1)**: The trajectories of the image features show an inefficient path towards the desired locations, with noticeable oscillations and deviations, indicating suboptimal control behavior.
- **Feature Error Norm (Figure 2)**: The feature error initially increases, likely due to improper initial conditions or high sensitivity, before converging. This suggests instability at the start, followed by a slower convergence phase.
- **Camera Pose Over Iterations (Figure 3)**: The camera undergoes large orientation changes before stabilizing. The position also exhibits slow adaptation, reflecting a non-smooth control process.
- **Camera Trajectory (Figure 4)**: The camera follows a highly curved trajectory, suggesting inefficient motion planning with unnecessary detours before reaching the target.
- **Camera Velocity Over Iterations (Figure 5)**: The translational and rotational velocities fluctuate significantly, indicating unstable control gains and potential overshooting.
- **Joint Angles Over Time (Figure 6)**: The joint angles undergo sharp transitions, highlighting abrupt corrections in the control strategy rather than smooth adjustments.

- **Joint Velocities Over Iterations (Figure 7)**: Some joint velocities exhibit extreme spikes, particularly in early iterations, suggesting sudden and excessive corrections that could lead to mechanical stress.

4.2.2 Case 2: Orientation Mismatch [10°, 30°, 60°]

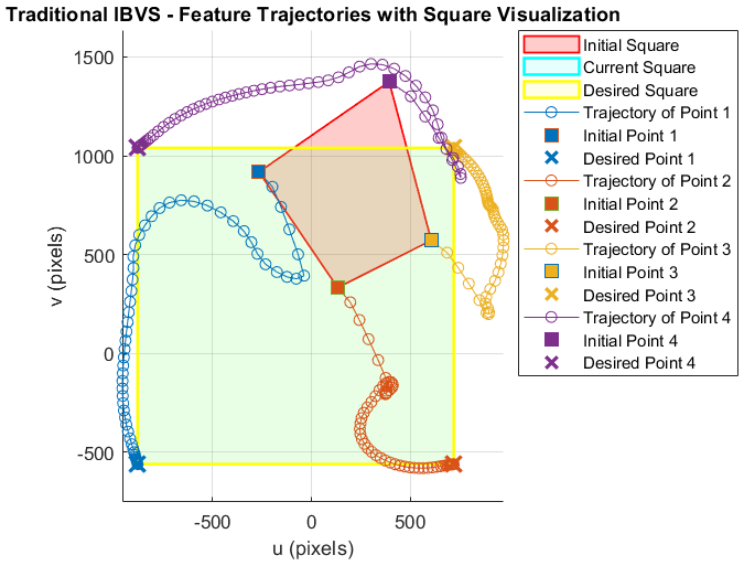

Figure 8: Traditional IBVS - Case 2: Feature Trajectories

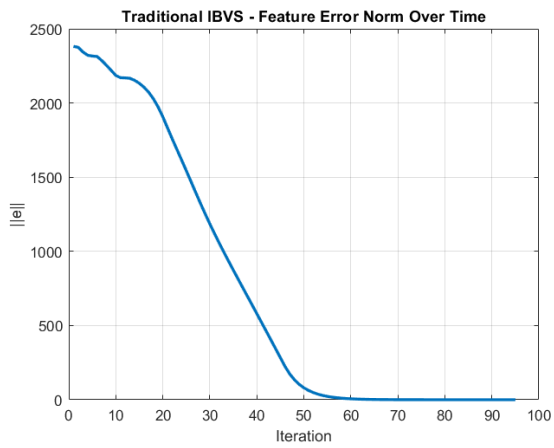

Figure 9: Traditional IBVS - Case 2: Feature Error Norm Over Iterations

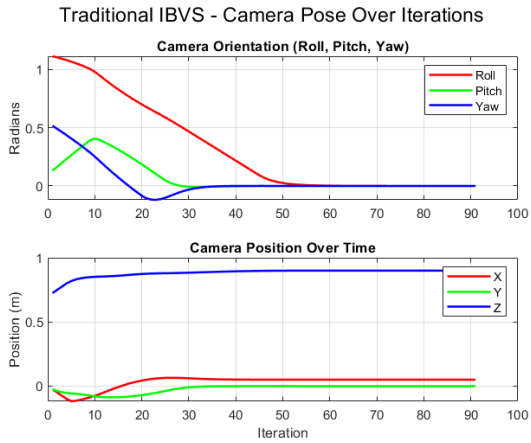

Figure 10: Traditional IBVS - Case 2: Camera Pose Over Iterations

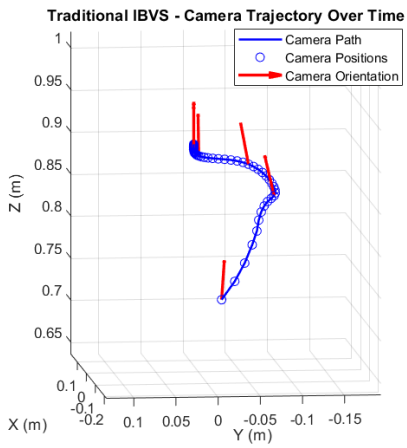

Figure 11: Traditional IBVS - Case 2: Camera Trajectory

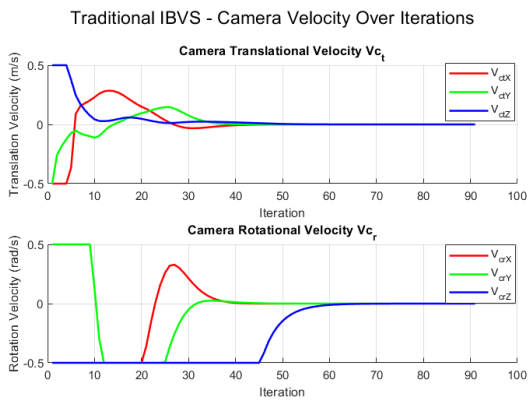

Figure 12: Traditional IBVS - Case 2: Camera Velocity Over Iterations

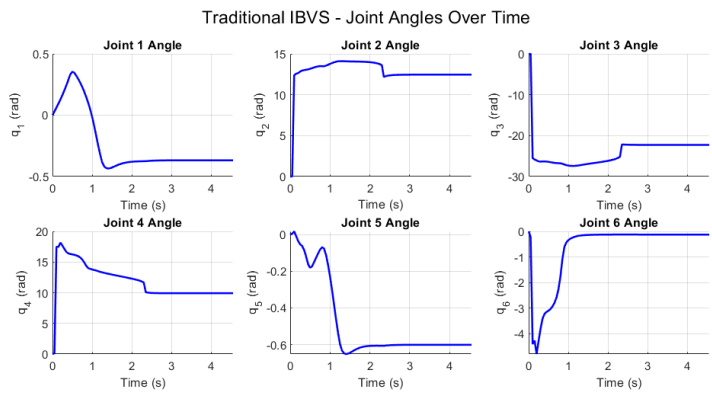

Figure 13: Traditional IBVS - Case 2: Joint Angles Over Time

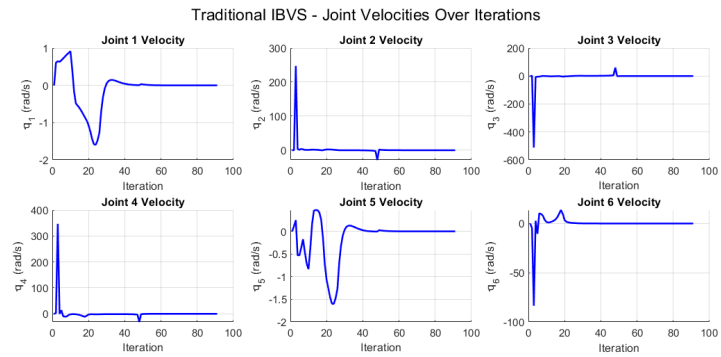

Figure 14: Traditional IBVS - Case 2: Joint Velocities Over Iterations

Explanation of Plots

- **Feature Trajectories (Figure 8)**: This plot illustrates the movement of the image features throughout the IBVS process. The initial, current, and desired positions of the tracked features are visualized along with their trajectories. The feature points exhibit curved paths due to the coupled rotational and translational motions of the camera. The convergence of feature points towards the desired positions confirms the successful execution of the control scheme.

- **Feature Error Norm (Figure 9)**: This graph displays the norm of the feature error over time. A decreasing trend in the error norm indicates successful feature alignment with the desired configuration. The plot suggests an initially high error, which gradually reduces as the control scheme corrects the feature positions, confirming effective visual servoing.

- **Camera Pose Over Iterations (Figure 10)**: This plot presents the evolution of the camera's orientation (roll, pitch, yaw) and position (X, Y, Z) over time. The changes in orientation and translation suggest a complex trajectory required to align the camera with the desired viewpoint. The smooth convergence of these parameters towards stable values indicates successful servoing without oscillatory behavior.

- **Camera Trajectory (Figure 11)**: The 3D trajectory of the camera highlights its movement through space while correcting feature errors. The path demonstrates

how the camera moves from its initial pose towards the desired position while maintaining visual feature tracking. The final position aligns with the desired configuration, demonstrating effective convergence.

- **Camera Velocity Over Iterations (Figure 12)**: This plot depicts the camera's translational and rotational velocities throughout the IBVS process. High initial velocity values suggest rapid corrections at the beginning, followed by smooth decay as the system stabilizes. The control scheme effectively damps oscillations, ensuring stable convergence.
- **Joint Angles Over Time (Figure 13)**: The evolution of joint angles indicates how the robot manipulator adjusts its configuration to achieve the desired visual features. Large variations in some joints suggest significant reconfiguration, whereas steady values towards the end confirm successful convergence.
- **Joint Velocities Over Iterations (Figure 14)**: This plot displays the velocity profiles of each joint. The presence of high velocities in the early phase suggests an aggressive response to initial feature errors. As time progresses, the velocities stabilize, ensuring smooth motion without abrupt changes.

4.2.3 Case 3: Orientation Mismatch [10°, 15°, 200°]

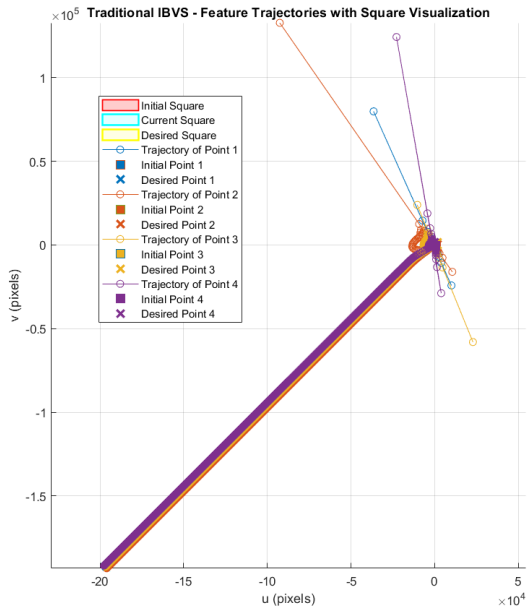

Figure 15: Traditional IBVS - Case 3: Feature Trajectories

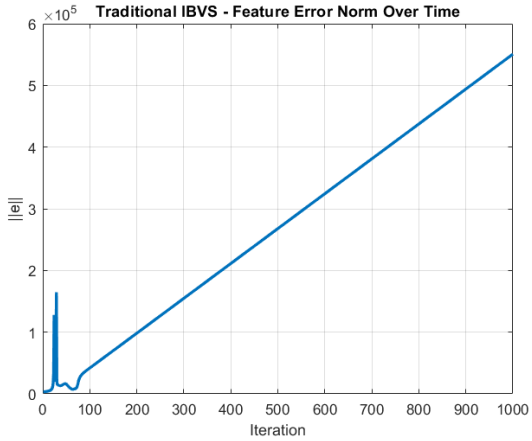

Figure 16: Traditional IBVS - Case 3: Feature Error Norm Over Iterations

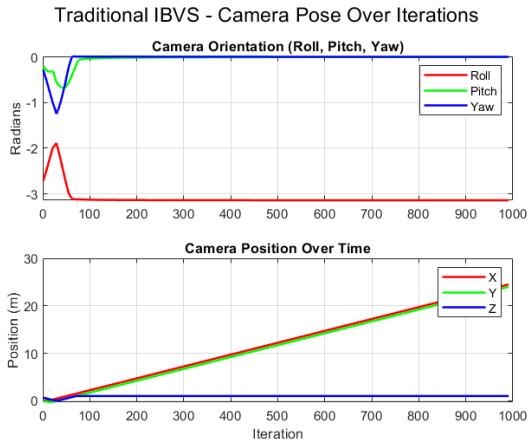

Figure 17: Traditional IBVS - Case 3: Camera Pose Over Iterations

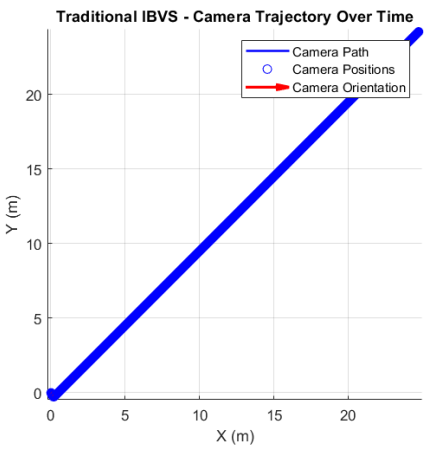

Figure 18: Traditional IBVS - Case 3: Camera Trajectory

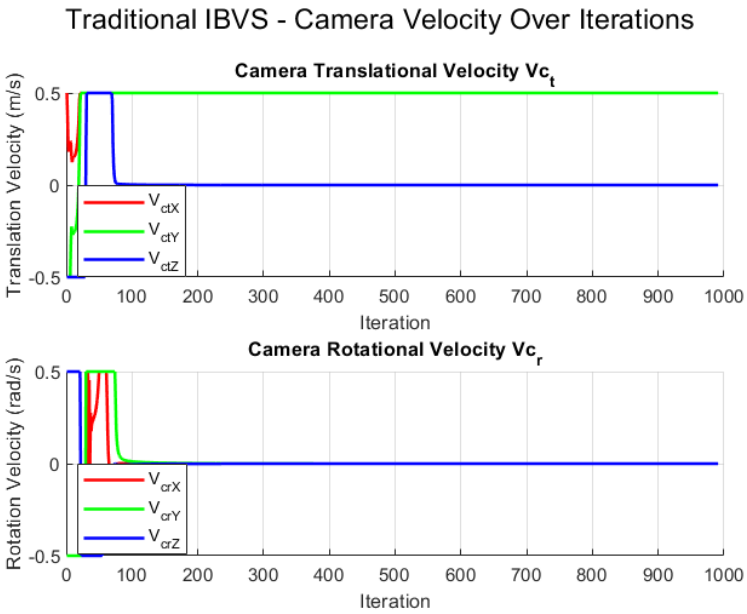

Figure 19: Traditional IBVS - Case 3: Camera Velocity Over Iterations

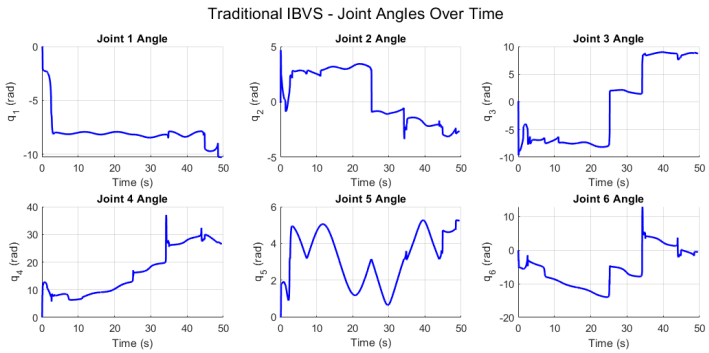

Figure 20: Traditional IBVS - Case 3: Joint Angles Over Time

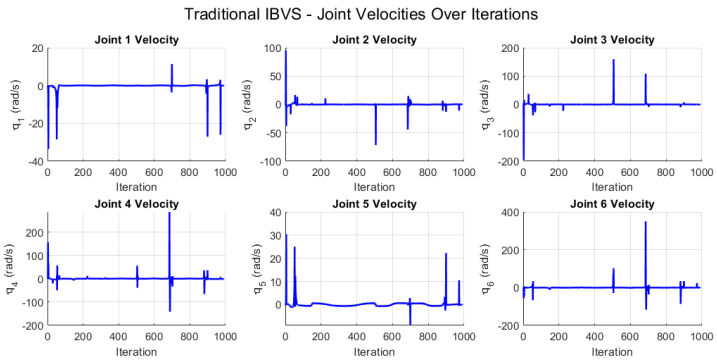

Figure 21: Traditional IBVS - Case 3: Joint Velocities Over Iterations

Explanation of Plots In Case 3, the Traditional IBVS method failed to converge within 1000 iterations, as evidenced by the persistent feature error norm (Figure 16). The feature trajectories and camera pose plots (Figure 15, Figure 17) illustrate the inability of the controller to align the camera's orientation effectively under extreme mismatches.

4.3 Adaptive Switch IBVS Implementation Results

This subsection showcases the results obtained from the Adaptive Switch IBVS method, emphasizing its enhanced capability to handle larger orientation mismatches and achieve faster convergence. Eight plots are generated for each test case, encompassing feature trajectories, error norms, switching angles, camera poses, camera trajectories, camera velocities, joint angles, and joint velocities over iterations.

4.3.1 Case 1: Orientation Mismatch [30°, 30°, 30°]

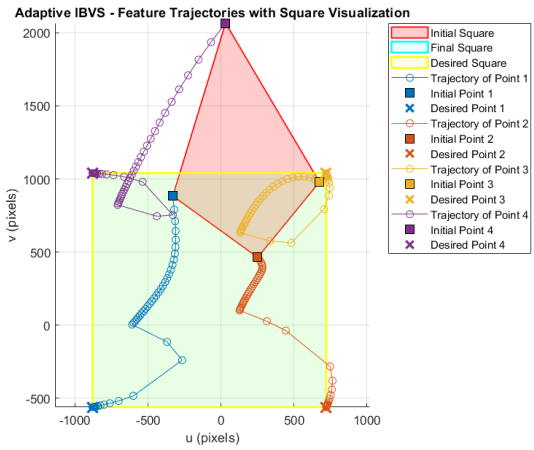

Figure 22: Adaptive IBVS - Case 1: Feature Trajectories

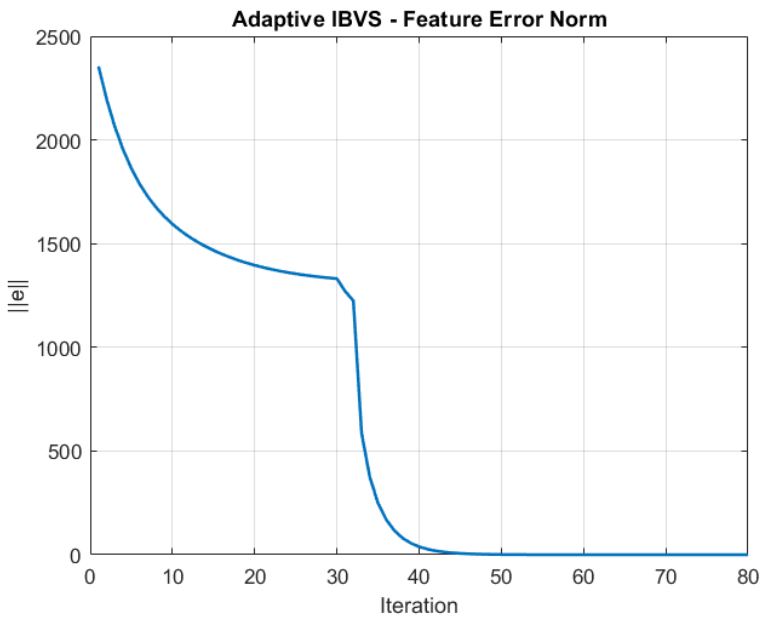

Figure 23: Adaptive IBVS - Case 1: Feature Error Norm Over Iterations

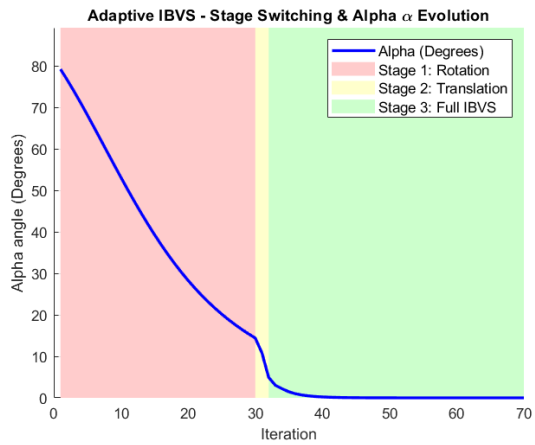

Figure 24: Adaptive IBVS - Case 1: Alpha Angle Evolution

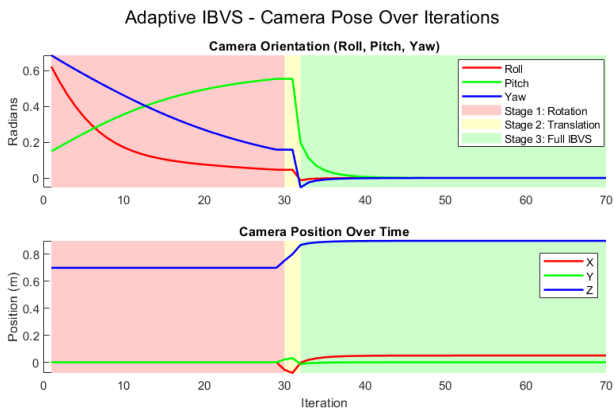

Figure 25: Adaptive IBVS - Case 1: Camera Pose Over Iterations

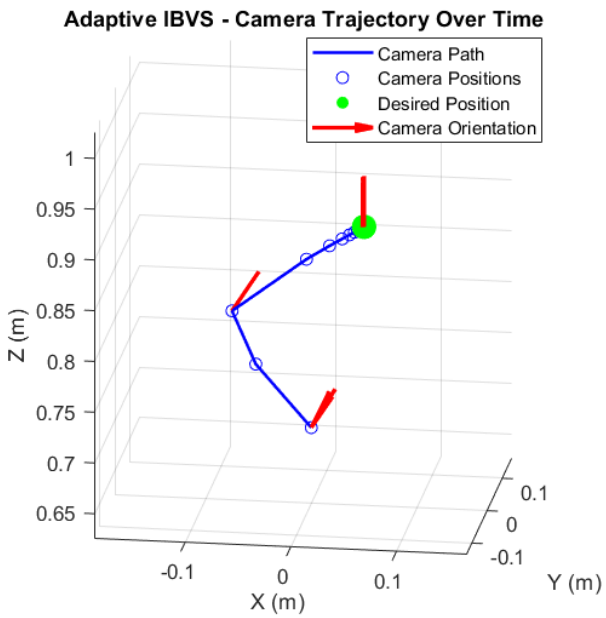

Figure 26: Adaptive IBVS - Case 1: Camera Trajectory

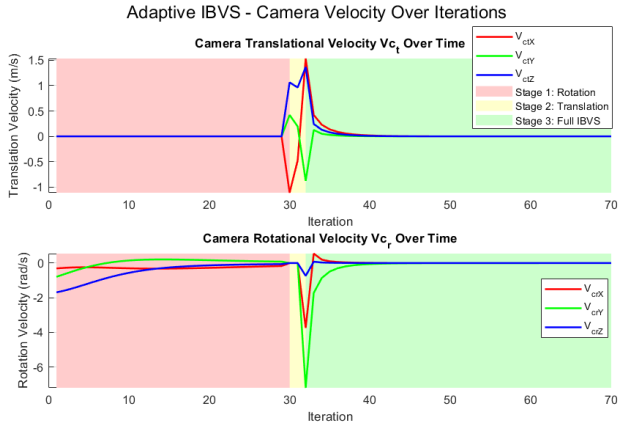

Figure 27: Adaptive IBVS - Case 1: Camera Velocity Over Iterations

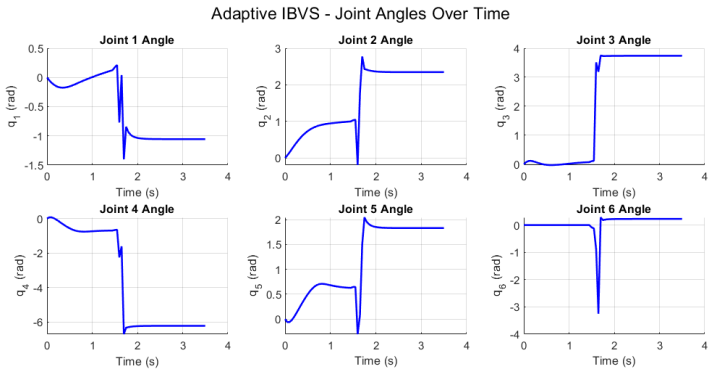

Figure 28: Adaptive IBVS - Case 1: Joint Angles Over Time

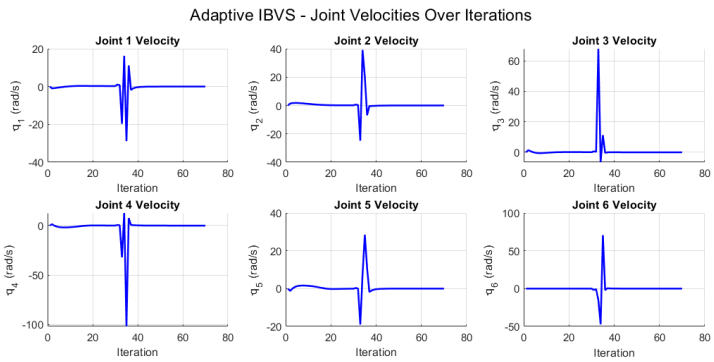

Figure 29: Adaptive IBVS - Case 1: Joint Velocities Over Iterations

Explanation of Plots

- **Feature Trajectories (Figure 22)**: The initial (red) and desired (yellow) squares illustrate the starting and target feature points. The adaptive IBVS guides the feature points smoothly, with noticeable curved trajectories due to dynamic control adjustments.
- **Feature Error Norm (Figure 23)**: The error norm decreases significantly, with a sharp drop around iteration 30, indicating the transition from rotation correction to full IBVS. The rapid convergence highlights the effectiveness of adaptive switching.

- **Stage Switching & Alpha Evolution (Figure 24)**: The system starts in Stage 1 (rotation), then transitions through Stage 2 (translation) before reaching Stage 3 (full IBVS) around iteration 35. The structured transition ensures stable convergence.

- **Camera Pose Over Iterations (Figure 25)**: The camera orientation (roll, pitch, yaw) is initially corrected aggressively, while the position remains stable in early iterations. Once full IBVS is engaged, translation adjustments complete the convergence.

- **Camera Trajectory (Figure 26)**: The 3D trajectory shows that the initial motion is primarily rotational, with translation occurring later. The adaptive IBVS ensures a smooth approach to the desired position.

- **Camera Velocity Over Iterations (Figure 27)**: Rotational velocities dominate early iterations, while translation velocities increase sharply after transitioning to full IBVS. The system stabilizes in later iterations.

- **Joint Angles Over Time (Figure 28)**: Some joint angles undergo abrupt changes, particularly during the transition between stages. The final angles align with the desired pose.

- **Joint Velocities Over Iterations (Figure 29)**: Velocity spikes occur during stage transitions, reflecting the adaptive gain adjustments. The velocities stabilize as the system converges.

4.3.2 Case 2: Orientation Mismatch [10°, 30°, 60°]

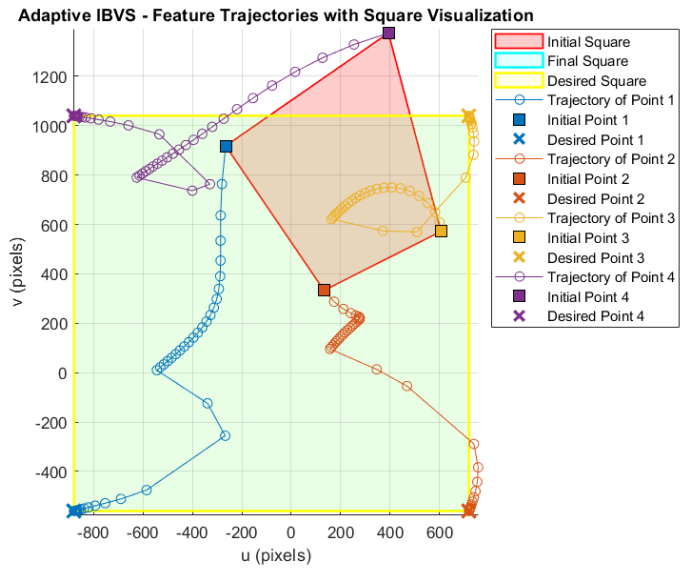

Figure 30: Adaptive IBVS - Case 2: Feature Trajectories

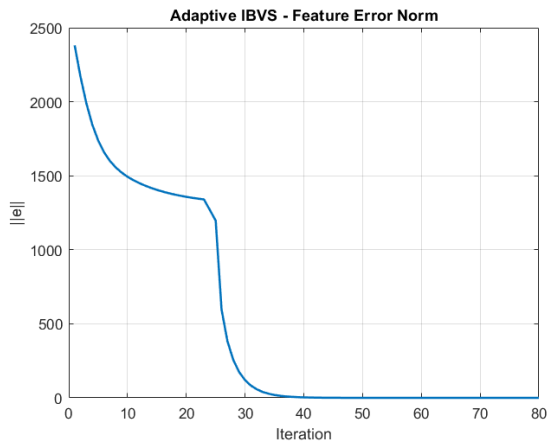

Figure 31: Adaptive IBVS - Case 2: Feature Error Norm Over Iterations

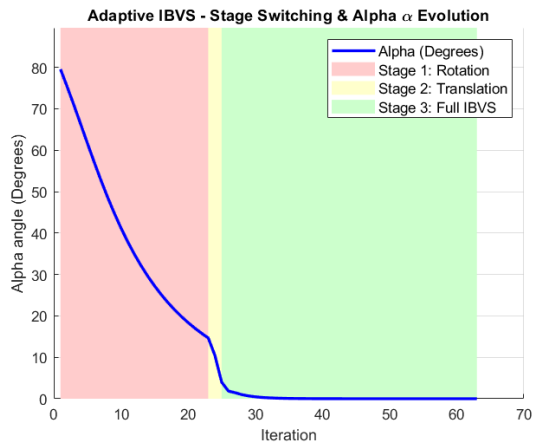

Figure 32: Adaptive IBVS - Case 2: Alpha Angle Evolution

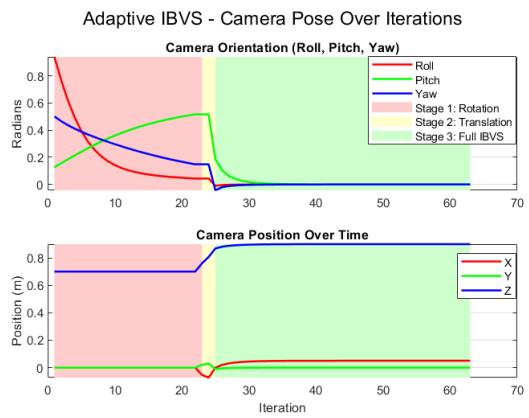

Figure 33: Adaptive IBVS - Case 2: Camera Pose Over Iterations

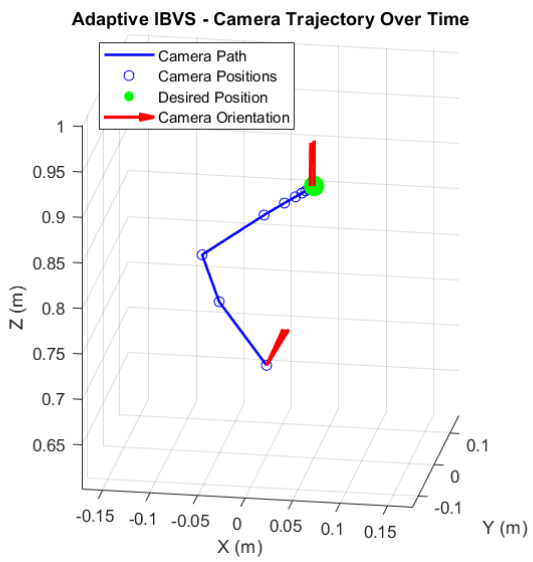

Figure 34: Adaptive IBVS - Case 2: Camera Trajectory

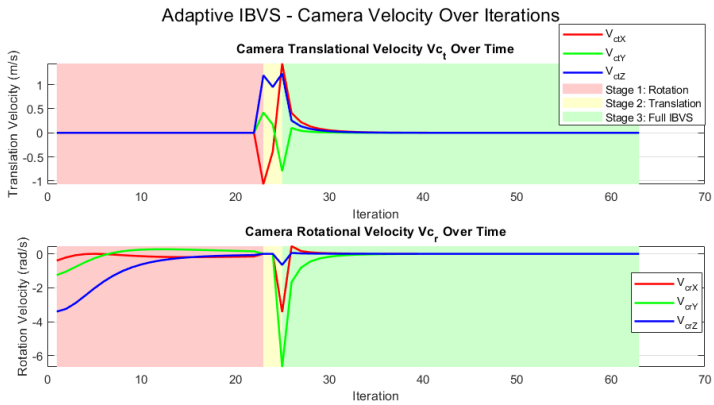

Figure 35: Adaptive IBVS - Case 2: Camera Velocity Over Iterations

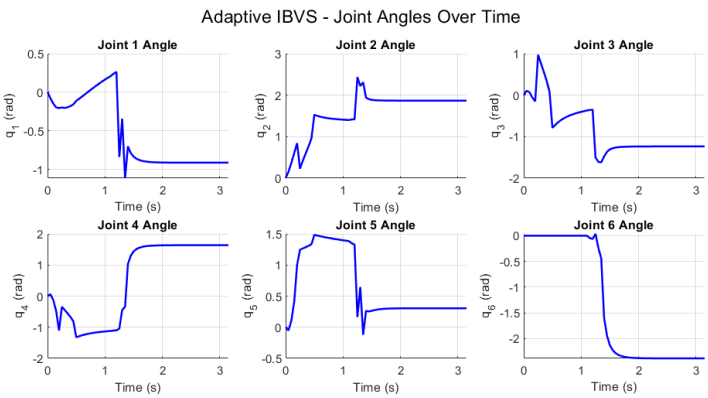

Figure 36: Adaptive IBVS - Case 2: Joint Angles Over Time

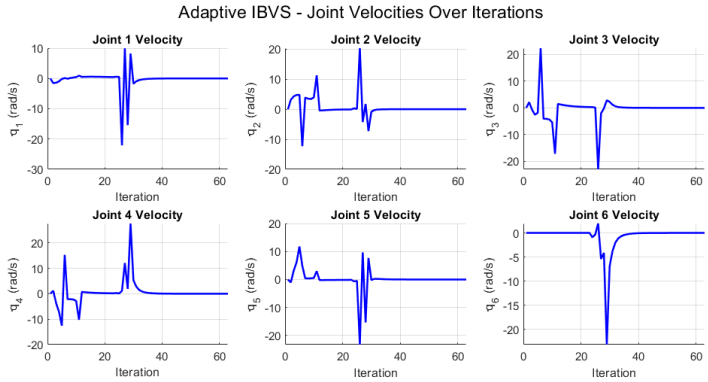

Figure 37: Adaptive IBVS - Case 2: Joint Velocities Over Iterations

Explanation of Plots

- **Feature Trajectories (Figure 30)**: The feature points start from a highly misaligned configuration and smoothly move toward the desired positions. The adaptive strategy effectively ensures a stable trajectory, reducing large deviations during the transition.
- **Feature Error Norm (Figure 31)**: The error norm decreases in a staged manner, first at a moderate rate during rotation correction, followed by a steeper descent as the translation and full IBVS phases are activated.
- **Stage Switching and Alpha Evolution (Figure 32)**: The alpha value starts at a high initial value and gradually decreases, triggering the transitions at predefined thresholds. The clear phase distinctions indicate smooth switching between control modes.
- **Camera Pose Over Iterations (Figure 33)**: The roll, pitch, and yaw angles adjust significantly in the early phase, confirming the priority of orientation correction before position control. The position changes more significantly after rotation stabilizes.
- **Camera Trajectory (Figure 34)**: The camera moves along a controlled path towards the desired position, following a structured approach. The trajectory shows a distinct change in movement characteristics at different stages.
- **Camera Velocity Over Iterations (Figure 35)**: The rotational velocity is dominant initially, corresponding to the rotation stage, while the translational velocity increases as the control transitions to position-based IBVS.
- **Joint Angles Over Time (Figure 36)**: The joint angles demonstrate a smooth but reactive adaptation to the switching control strategy, particularly around the transition points. The joint movements stabilize once the camera reaches its goal.
- **Joint Velocities Over Iterations (Figure 37)**: The joint velocity profiles show clear peaks around the transitions, reflecting the controller's effort in adjusting for different stages. The velocities stabilize as the system converges.

4.3.3 Case 3: Orientation Mismatch [10°, 15°, 200°]

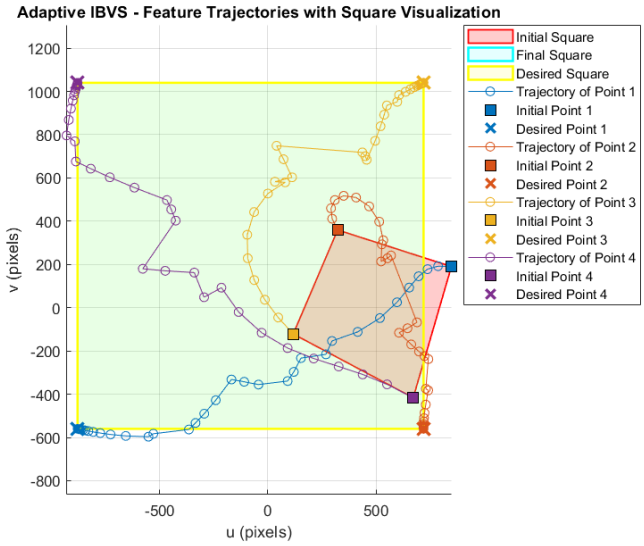

Figure 38: Adaptive IBVS - Case 3: Feature Trajectories

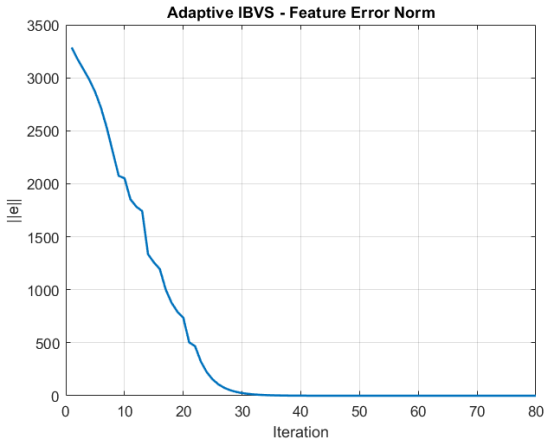

Figure 39: Adaptive IBVS - Case 3: Feature Error Norm Over Iterations

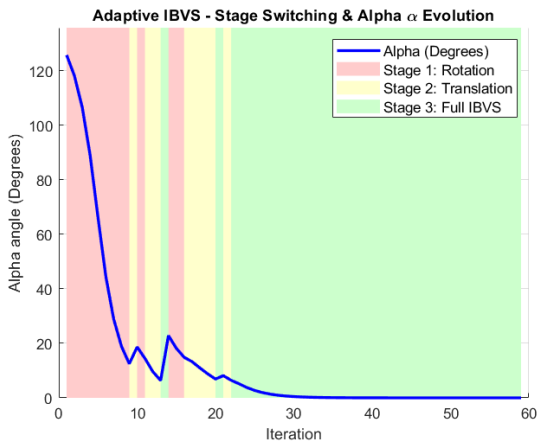

Figure 40: Adaptive IBVS - Case 3: Alpha Angle Evolution

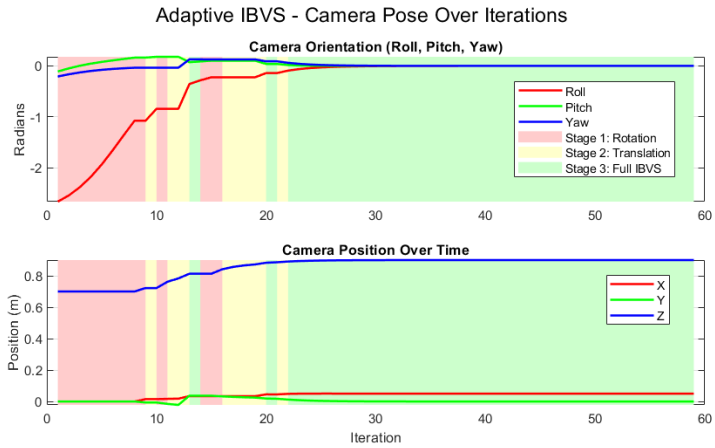

Figure 41: Adaptive IBVS - Case 3: Camera Pose Over Iterations

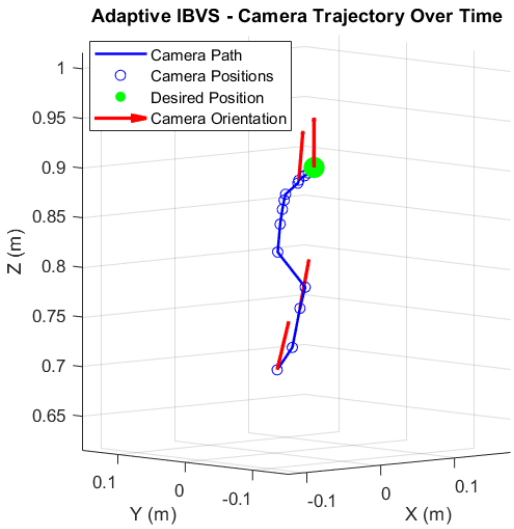

Figure 42: Adaptive IBVS - Case 3: Camera Trajectory

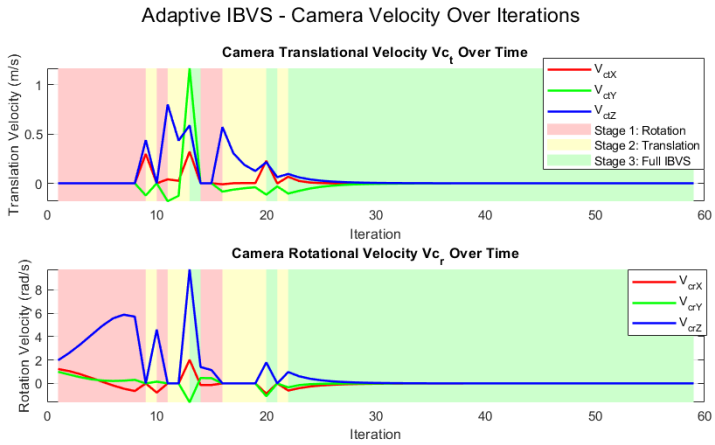

Figure 43: Adaptive IBVS - Case 3: Camera Velocity Over Iterations

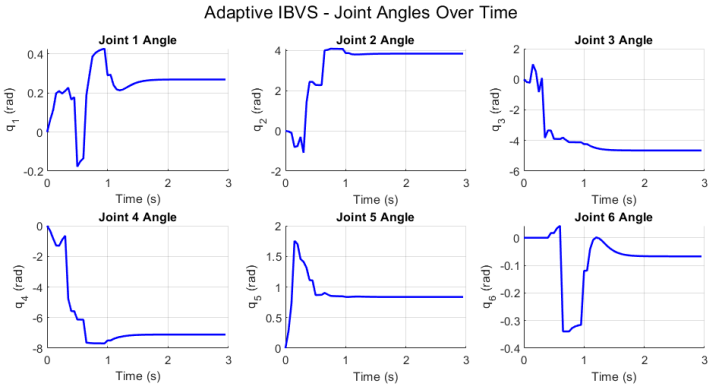

Figure 44: Adaptive IBVS - Case 3: Joint Angles Over Time

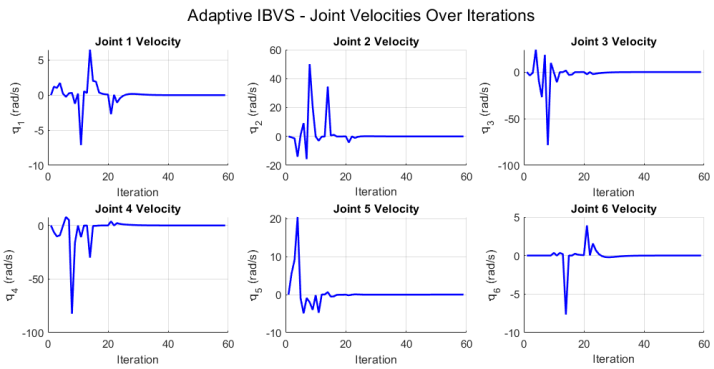

Figure 45: Adaptive IBVS - Case 3: Joint Velocities Over Iterations

Explanation of Plots

- **Feature Trajectories (Figure 38)**: The trajectories of the four feature points show a smooth convergence towards their desired positions. The initial large displacement is effectively corrected as the control process progresses, leading to minimal final error.

- **Feature Error Norm (Figure 39)**: The feature error norm decreases consistently, indicating successful minimization of the visual error. The sharp decline at the beginning reflects rapid corrections during the initial phase.
- **Camera Pose Over Iterations (Figure 41)**: The camera orientation (roll, pitch, yaw) shows a gradual stabilization after initial fluctuations, while the position variables converge smoothly to their final values, ensuring correct alignment with the desired pose.
- **Camera Trajectory (Figure 42)**: The 3D trajectory reveals a smooth movement towards the target pose. The path follows a structured motion with clear corrections to orientation as indicated by the red arrows.
- **Camera Velocity Over Iterations (Figure 43)**: Both translational and rotational velocities exhibit oscillations in the early iterations, which gradually dampen as the system reaches convergence, demonstrating the adaptive nature of the control.
- **Joint Angles Over Time (Figure 44)**: The joint angles undergo significant variations during the initial adaptation phase before settling into stable values, ensuring proper alignment of the robotic system.
- **Joint Velocities Over Iterations (Figure 45)**: The joint velocities display sharp transients at the start, followed by a gradual stabilization, confirming the smooth execution of the control strategy.

4.3.4 Extreme Case: Orientation Mismatch [200°, 110°, -130°]

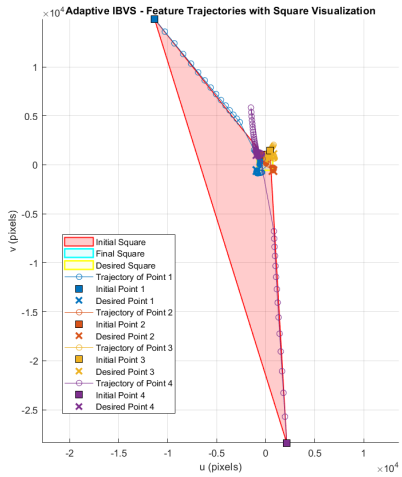

Figure 46: Adaptive IBVS - Extreme Case: Feature Trajectories

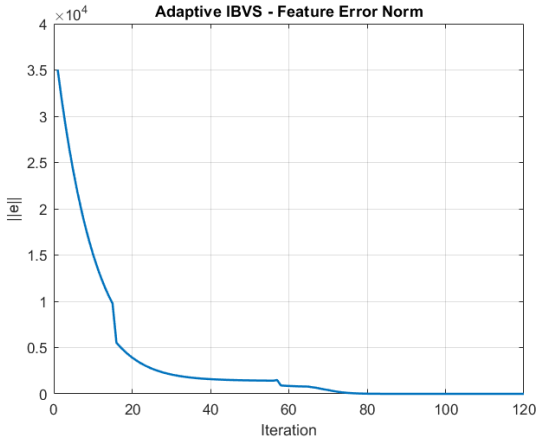

Figure 47: Adaptive IBVS - Extreme Case: Feature Error Norm Over Iterations

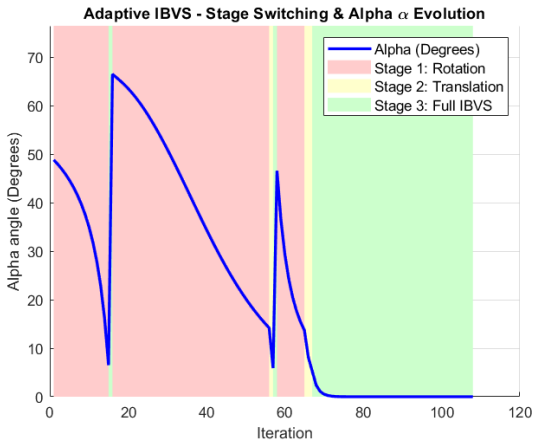

Figure 48: Adaptive IBVS - Extreme Case: Alpha Angle Evolution

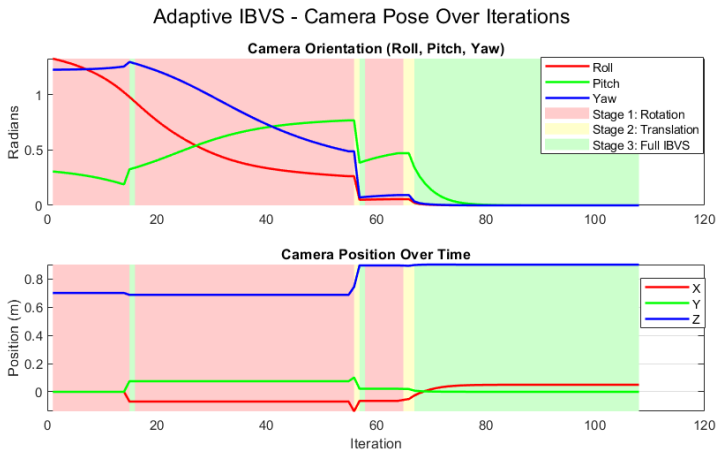

Figure 49: Adaptive IBVS - Extreme Case: Camera Pose Over Iterations

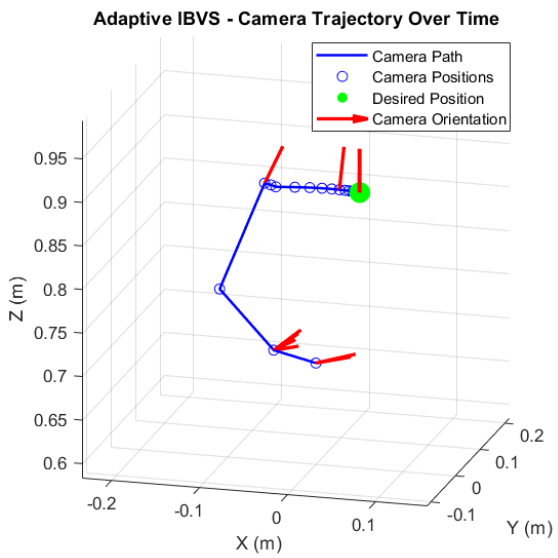

Figure 50: Adaptive IBVS - Extreme Case: Camera Trajectory

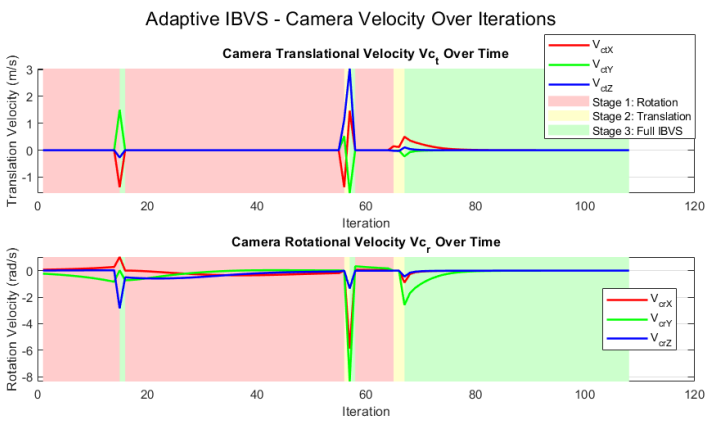

Figure 51: Adaptive IBVS - Extreme Case: Camera Velocity Over Iterations

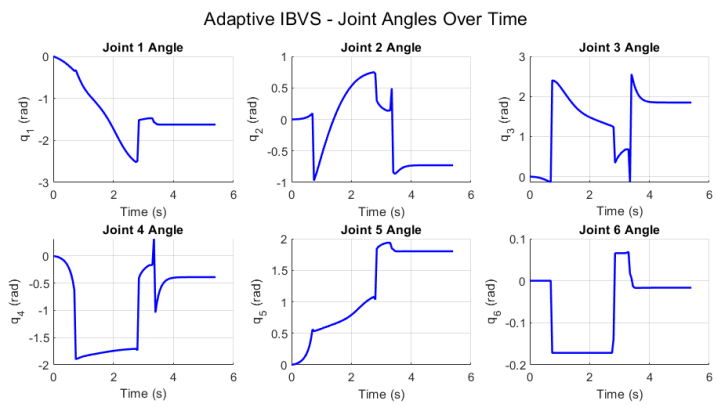

Figure 52: Adaptive IBVS - Extreme Case: Joint Angles Over Time

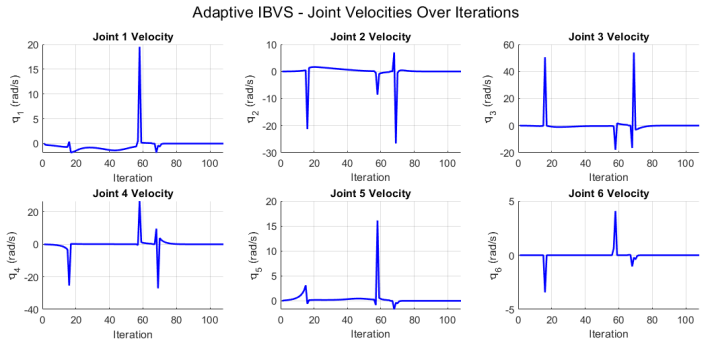

Figure 53: Adaptive IBVS - Extreme Case: Joint Velocities Over Iterations

Explanation of Plots

- **Feature Trajectories (Figure 46)**: The feature points start from a highly distorted configuration and follow a curved trajectory before converging to the desired square. The large initial displacement indicates significant misalignment, which is progressively corrected.
- **Feature Error Norm (Figure 47)**: The error norm exhibits an initially high value, reflecting the extreme misalignment of features. The gradual decline with minor fluctuations suggests adaptive corrections before reaching convergence.
- **Camera Pose Over Iterations (Figure 49)**: The camera undergoes substantial rotational and translational adjustments, particularly in the early iterations. The final pose remains stable, indicating successful convergence.
- **Camera Trajectory (Figure 50)**: The camera follows a complex, non-linear trajectory, reflecting the significant adjustments needed for alignment. The final approach to the target position confirms the effectiveness of the adaptive IBVS.
- **Camera Velocity Over Iterations (Figure 51)**: The velocity profile shows sharp peaks during the transition phases, highlighting significant corrections in motion. As convergence is reached, velocity stabilizes close to zero.
- **Joint Angles Over Time (Figure 52)**: The joint angles exhibit substantial variations, particularly in the early stages, as the manipulator adapts to extreme misalignment. The final steady-state values indicate successful convergence.
- **Joint Velocities Over Iterations (Figure 53)**: The joint velocities show large fluctuations, especially in the early stages, reflecting the effort required for correction. The eventual damping of oscillations indicates stability.
- **Stage Switching & Alpha Evolution (Figure 48)**: The alpha angle undergoes sharp variations as the system adapts dynamically. The transitions between stages are more frequent due to the extreme initial misalignment, requiring multiple corrective adjustments.

4.4 Comparative Analysis

Comparing the results of the Traditional IBVS and Adaptive Switch IBVS across all test cases reveals significant improvements in the adaptive approach. The Adaptive Switch IBVS consistently achieved convergence in fewer iterations and was capable of handling extreme orientation mismatches where the Traditional IBVS failed. The staged control mechanism, governed by the alpha angle thresholds α_0 and α_1, played a crucial role in enhancing the controller's responsiveness and robustness.

Table 1: Convergence Summary of IBVS Methods

Test Case	**Traditional IBVS (Iterations)**	**Adaptive IBVS (Iterations)**
[30°, 30°, 30°]	96	70
[10°, 30°, 60°]	95	63
[10°, 15°, 200°]	**Failed**	59
Extreme Case: [200°, 110°, -130°]	**Failed**	108

Insights from the Comparative Analysis The table above (Table 1) succinctly summarizes the convergence performance of both IBVS methods across various orientation mismatches. The Adaptive Switch IBVS not only converges faster but also successfully handles scenarios where the Traditional IBVS fails to achieve convergence within the stipulated iteration limit. This enhancement underscores the effectiveness of the adaptive switching mechanism in mitigating the limitations of conventional IBVS, particularly in managing significant orientation discrepancies.

5 Conclusion

The implementation and analysis of the Adaptive Switch IBVS method within a MATLAB simulation environment reaffirm its superiority over the Traditional IBVS approach. By decoupling rotational and translational movements and employing an adaptive switching mechanism based on orientation error thresholds, the Adaptive Switch IBVS demonstrated enhanced convergence capabilities and robustness against large orientation mismatches. These findings align with the objectives outlined in Ghasemi et al.'s seminal work, validating the theoretical advancements through practical simulation. Future work may involve extending this implementation to real-world robotic systems and integrating dynamic modeling to further enhance control performance.

References

[1] A. Ghasemi, P. Li, and W.-F. Xie, *Adaptive Switch Image-Based Visual Servoing for Industrial Robots*, International Journal of Control, Automation and Systems, vol. 18, no. 5, pp. 1324-1334, 2020.

[2] R. Barbălat, *Stability of Nonlinear Systems*, SIAM Journal on Control, vol. 3, no. 3, pp. 438–449, 1965.

[3] F. Chaumette and S. Hutchinson, *Visual Servo Control. I. Basic Approaches*, IEEE Robotics & Automation Magazine, vol. 13, no. 4, pp. 82–90, 2006.

[4] F. Chaumette and S. Hutchinson, *Visual Servo Control - Part II: Advanced Approaches*, IEEE Robotics & Automation Magazine, vol. 14, no. 1, pp. 109–118, 2007.